



**Middlesex  
University  
London**

**MSc Individual Project Report**

**25.09.2023**

**Student Name:**

Waqas Salman

**Student Number:**

M00851440

**Supervisor Name:**

Chris Huyck

**Course:**

CST4444 Individual Project

**Developing an AI-Powered Chatbot for Fisheye Real  
Estate Stock Footage Productions**

*Dedicated to my family, especially my parents as well as my supervisor*

## **ACKNOWLEDGEMENTS**

I would like to thank and acknowledge my proposed supervisor **Chris Huyck**. He provided valuable suggestions that helped me to complete this report. Thank you, Sir, for providing your honest feedback whenever required. Without your support, it was not possible to complete this project.

I would also like to acknowledge my parents and family back home who supported me with their unconditional love while I was busy making this project. It is due to their sincere support and prayers that I was able to complete my dissertation.

Last but not least, I would like to acknowledge my batch mates at Middlesex University who assisted me with their valuable suggestions and advice that acted as a phenomenal asset for this project and I owe them for this.

**Signed By:** Waqas Salman

**25th September 2023**

## DECLARATION

I, Waqas Salman, Candidate for MSc in Software Engineering Programme at Middlesex University, hereby declare that the matter printed in this report is solely my own work and has not been printed, published and/or submitted as research work, dissertation or publication in any form across any university in the United Kingdom or other foreign academic institutions.

**Signed By:** Waqas Salman

## **EXECUTIVE SUMMARY**

This individual report features the development of a chatbot for a client who has an online stock footage production website called “Fisheye”. This website offers its clients a paid subscription that facilitates them with high-resolution images and graded videos of property locations. The idea was to create a functional chatbot that could fully assist the subscribed clients in finding their desired images and footage as and when required. For this purpose, the chatbot was developed using the Natural Language Processing Toolkit and Keras Libraries in Python. All the development stages, implementation, documentation, and final evaluation of the chatbot are covered in detail in this report. Once developed, the chatbot files were shared with the client and he was extremely satisfied with the functionality and responses of the chatbot, which will be made live by the client soon.

# TABLE OF CONTENTS

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| <b>Chapter-1: Introduction .....</b>                                  | <b>1</b>  |
| Introduction to Fisheye Real Estate Stock Footage Productions .....   | 1         |
| Important Concepts & Techniques Used in Building The Chatbot.....     | 1         |
| Project Structure .....                                               | 2         |
| Project Objectives .....                                              | 3         |
| Project Methodology.....                                              | 4         |
| <b>Chapter - 2: Literature Review .....</b>                           | <b>5</b>  |
| Research Related to The Need for an AI-Powered Chatbot .....          | 5         |
| Research Related to Chatbot Development .....                         | 6         |
| Challenges Associated with Chatbot Development .....                  | 9         |
| <b>Chapter - 3: Implementation.....</b>                               | <b>12</b> |
| WordNet and Synsets .....                                             | 15        |
| Similarity Comparison .....                                           | 15        |
| Tree and Treebank.....                                                | 16        |
| Named Entity Recognition (NER) .....                                  | 16        |
| <b>Chapter - 4: Project Work Evaluation Strategy / Test Plan.....</b> | <b>18</b> |
| App.py.....                                                           | 21        |
| Flask Code for Running and Rendering HTML Data.....                   | 22        |
| Training the File for Chatbot Understanding - Training.py .....       | 23        |
| Chatbot HTML Code .....                                               | 25        |
| Data File.....                                                        | 25        |
| <b>Chapter - 5: Documentation .....</b>                               | <b>27</b> |
| Technical Documentation .....                                         | 27        |
| User Documentation - How to Run The Chatbot.....                      | 28        |
| <b>Chapter - 6: Evaluation .....</b>                                  | <b>33</b> |
| Overview of the Project .....                                         | 33        |
| Performance Metrics .....                                             | 33        |
| Advantages of The Chatbot.....                                        | 34        |
| Disadvantages of the Chatbot .....                                    | 35        |
| Discussion.....                                                       | 35        |
| Conclusion and Recommendations .....                                  | 40        |
| <b>References .....</b>                                               | <b>43</b> |
| <b>Appendix .....</b>                                                 | <b>48</b> |
| Documenting the Code Development for Chatbot .....                    | 48        |

# LIST OF FIGURES

## **Chapter - 4: Project Work Evaluation Strategy / Test Plan**

|                                                                                         |    |
|-----------------------------------------------------------------------------------------|----|
| An overview of an ongoing chat between the user and the chatbot                         | 18 |
| A further overview of an ongoing chat between the user and chatbot                      | 19 |
| An overview of the homepage of the Fisheye website from where the chatbot is accessible | 20 |
| An overview of the main landing page of the Fisheye website                             | 20 |
| An overview of App.py back-end coding                                                   | 21 |
| A further overview of App.py back-end coding                                            | 21 |
| An overview of Flask backend coding                                                     | 22 |
| An overview of Training.py back-end coding                                              | 24 |
| A further overview of Training.py back-end coding                                       | 24 |
| An overview of model coding after the completion of Training                            | 25 |
| An overview of Integration of HTML Code with App.py and API                             | 25 |
| An overview of Data Files stored in the form of JSON Files                              | 26 |

## **Chapter - 5: Documentation**

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| An overview of the Front-end Desktop view                                         | 29 |
| A browser overview from where the user can access the Fisheye website             | 29 |
| An overview of the website interface featuring the search bar                     | 30 |
| The “Chat Now” box from where the user can start its interaction with the chatbot | 30 |
| The first message of the chatbot from where the conversation begins               | 31 |
| An overview of Survey Monkey Feedback Form Integrated with the Chatbot            | 32 |

## **Chapter - 6: Evaluation**

|                                                                     |    |
|---------------------------------------------------------------------|----|
| Conversation with the Client regarding Chatbot Functionality        | 36 |
| Survey Monkey Insights                                              | 36 |
| Statistical Preview of User Response to Feedback Questionnaire - Q1 | 37 |
| Statistical Preview of User Response to Feedback Questionnaire - Q2 | 37 |
| User Response to Open Ended Questions like Q3                       | 38 |
| Statistical Preview of User Response to Feedback Questionnaire - Q4 | 38 |

# **Chapter-1: Introduction**

This project features the development of a chatbot for Fisheye Real Estate Stock Footage Productions. Powered by Artificial Intelligence (AI), this chatbot has the capability of enhancing the subscription experience of the stock footage website. Leveraging the power of AI, this chatbot is equipped with a comprehensive dataset of high-resolution images and graded videos of various property locations. The project aims to streamline the stock footage selection process by making it easier for subscribed clients to find stock footage with their required description. A user-friendly and interactive interface serves as a foundation for this enhanced client interaction. Moreover, advanced AI and programming frameworks (NLP, NLTK, and Keras Libraries in Python) were used during the development process of the chatbot.

## **Introduction to Fisheye Real Estate Stock Footage Productions**

Fisheye Real Estate Stock Footage Productions is a specialized company dedicated to assisting potential real estate clients in their property search journey. They offer a comprehensive suite of visually compelling resources to aid in the decision-making process. One of their primary functions is to offer paid subscriptions to clients that unlock a variety of features for them including downloadable graded videos and high-resolution actual images of property locations. They also offer subscribed clients to put a logo or watermark on the images and videos available on the platform. They serve as a valuable bridge, helping clients navigate the often complex real estate landscape.

Furthermore, the company also specialises in creating captivating property images and videos. These visual assets play a crucial role in real estate marketing, allowing clients to explore properties and gain a vivid sense of their features, layout, and overall ambience. Ultimately, the company has the expertise in producing compelling visual content, empowering potential clients to make informed decisions about their real estate ventures.

## **Important Concepts & Techniques Used in Building The Chatbot**

This section focuses on some essential elements that underpin the creation of effective chatbots. It features various concepts and techniques including Natural Language Processing (NLP), Natural Language Toolkit (NLTK) and Keras Libraries in Python. Therefore, these concepts and techniques will be discussed in detail, unravelling the intricacies of chatbot construction.

NLP is a branch of AI that focuses on enabling computers to understand, interpret, and generate human language. It involves the interaction between computers and human

language, encompassing tasks like language translation, sentiment analysis, and speech recognition. NLP plays a crucial role in chatbot development by allowing bots to comprehend and generate text in a more human-like way. Chatbots leverage NLP techniques to process user inputs, extract meaning, and provide relevant responses. This technology enables chatbots to engage in meaningful and contextually accurate conversations, enhancing the user experience and making interactions more natural and intuitive (Meshram et al., 2021).

NLTK is a powerful Python library specifically developed to facilitate research and experimentation in NLP. NLTK provides tools and resources for various NLP tasks, including tokenization, stemming, part-of-speech tagging, and parsing (Solanki, 2021). In the context of chatbot development, NLTK offers a range of functionalities that assist in preprocessing and understanding text data. Chatbots can leverage NLTK's capabilities to clean and structure user inputs, extract essential information, and enhance language understanding accuracy. By integrating NLTK, developers can build chatbots with improved language processing abilities, resulting in more effective and context-aware conversations (Kathirvelu et al., 2022).

Keras is an open-source, high-level deep learning library that operates on top of popular deep learning frameworks such as TensorFlow and Theano (Brownlee, 2019). Its user-friendly interface allows developers to rapidly prototype and build neural network models. In the realm of chatbot development, Keras proves instrumental in creating and training complex language models. By simplifying the neural network design process, Keras enables developers to focus on crafting chatbot architectures tailored to NLP tasks. Whether it is implementing advanced language generation models or training intricate dialogue systems, Keras accelerates the development of chatbots by providing a flexible and intuitive framework for building and fine-tuning neural networks (Brownlee, 2019).

## **Project Structure**

The project unfolds through three essential phases, each intricately contributing to the development of an innovative AI-powered chatbot tailored for Fisheye Real Estate. The first step involves the construction of a chatbot with the capacity to seamlessly comprehend and process natural language. This pivotal capability enables the chatbot to understand user inquiries in a manner that closely mirrors human interaction.

Subsequently, the integration process takes centre stage as the chatbot connects to the expansive real-estate database. This vital link grants the chatbot access to an extensive repository of knowledge, empowering it with the information required to fulfil user queries accurately and comprehensively.

Finally, the project culminates in the strategic design of a user-friendly interface to facilitate seamless and effective interactions between clients and the chatbot. This interface serves as the bridge that enables users to harness the chatbot's capabilities effortlessly, ensuring an effective level of customer engagement and satisfaction.

## **Project Objectives**

This research project is driven by the following objectives that collectively aim to create an AI-powered chatbot to provide better client experiences:

1. **Designing a Functional Chatbot Prototype:** The primary goal of the project is to construct a functional chatbot that harnesses advanced computing concepts, cutting-edge frameworks, and AI networks. By integrating sophisticated technologies, the chatbot will not only showcase the potential of AI but also lay the groundwork for achieving subsequent objectives.
2. **Integrating Natural Language Understanding:** A pivotal facet of the project involves reinforcing the chatbot with the ability to understand and process natural language queries. This integration of advanced NLP concepts ensures that clients can interact with the chatbot in a manner that feels natural and intuitive. Such linguistic prowess enables the chatbot to interpret user inquiries accurately, facilitating effective communication and enhancing the overall user experience.
3. **Empowering with Comprehensive Real Estate Knowledge:** Another critical objective focuses on empowering the chatbot with an extensive and up-to-date knowledge base of the real estate industry. The chatbot's effectiveness in delivering relevant and accurate information to clients hinges on its capacity to draw from a comprehensive database of domain-specific insights. By seamlessly accessing pertinent details, market trends, and property information, the chatbot becomes a trusted and informed companion for users seeking real estate-related advice and data.
4. **Creating an Intuitive User Interface:** In the pursuit of seamless client interactions, the project emphasizes the creation of a user-friendly and intuitive interface. This interface serves as the gateway through which users access the chatbot's capabilities. By prioritizing usability, navigation, and visual appeal, the project aims to create an environment where clients can effortlessly engage with the chatbot, enabling smooth and productive interactions.

These objectives collectively represent a holistic approach to developing an AI-powered chatbot. Through the convergence of advanced computing concepts like NLP, comprehensive domain knowledge, and user-centric design, the project aspires to deliver a transformative solution that meets the identified objectives for client engagement and interaction within the realm of the real estate stock footage industry.

## **Project Methodology**

**Chatbot Design:** The chatbot's features and functionalities will be designed, outlining its role in facilitating communication, providing technical guidance, and assistance in planning.

**Prototype Development:** Leveraging state-of-the-art AI and NLP technologies, a functional prototype of the chatbot will be developed, integrating it with the existing production workflow for testing.

**Usability Testing:** Industry professionals will participate in usability testing sessions, where they will interact with the chatbot prototype. Feedback on usability, effectiveness, and overall user experience will be collected.

**Optimization and Refinement:** The chatbot will be refined based on user feedback, with iterative improvements made to enhance its performance, accuracy, and usability.

In the next section, the phenomenon of chatbot development for businesses like Fisheye real estate stock footage production will be discussed in the spotlight of secondary research, focusing on the contributions of past researchers for the respective subject.

## **Chapter - 2: Literature Review**

AI has become more widely used in the real estate industry in the last few years (Miljkovic et al., 2023). Real estate firms now rely on NLP to create fully automated chatbots that assist their clientele in finding accurate information regarding property locations and prices (Spring et al., 2022; Singh, 2023). AI-powered chatbots are now seen as a promising solution to enhance client engagement and streamline interactions (Yadav, 2022). Therefore, this literature review aims to explore the existing research and advancements in developing an AI-powered chatbot specifically tailored for Fisheye Real Estate Stock Footage Productions.

### **Research Related to The Need for an AI-Powered Chatbot**

Implementing a chatbot on Fisheye's website will ensure round-the-clock availability for website visitors, a crucial advantage in the real estate industry where inquiries can arise at any time. This 24/7 support ensures that potential clients can receive timely responses to their queries and engage with the company even beyond regular business hours, thus accommodating diverse schedules (Jenneboer et al., 2022).

Furthermore, chatbots excel at providing instant responses, greatly enhancing the overall user experience (Stoilova, 2021). Their ability to swiftly address user inquiries not only captures the attention of potential clients but also keeps them engaged on the website, effectively reducing bounce rates and increasing the likelihood of conversion (Dash and Bakshi, 2019).

Scalability is another key advantage of chatbots. Real estate businesses like Fisheye often encounter fluctuations in website traffic and inquiries, making it challenging to ensure that no potential client is left waiting for assistance. Chatbots can manage multiple inquiries simultaneously, making them particularly valuable during peak times or when dealing with a high volume of inquiries (Torous et al., 2021). Moreover, chatbots efficiently retrieve and deliver detailed information about property listings, pricing, features, and more. This empowers potential clients with the data they need to make informed decisions, enhancing their confidence in the company's services (Seagraves, 2023).

Moreover, chatbots play a pivotal role in lead generation by collecting visitor information, such as contact details and preferences. These leads can be seamlessly passed to the sales or marketing team, streamlining the process of following up with interested prospects (Lee, 2020). Also, chatbots are a cost-effective alternative to hiring additional staff to handle inquiries around the clock. This cost-saving measure allows the company to allocate resources more efficiently and reduce labour costs (Nawaz and Saldeen, 2020).

Consistency in responses is another strength of chatbots. They ensure that all website visitors receive uniform and accurate information, contributing to the establishment of trust and credibility (Le, 2023). Additionally, chatbots engage visitors in interactive conversations, offering personalised recommendations based on user preferences. This interactive experience not only fosters engagement but also extends the time users spend on the website (Seagraves, 2023). Lastly, chatbots facilitate data collection and analysis by capturing user interactions and preferences. This data can be significant in understanding user behaviour and improving website content and services, ultimately enhancing the overall customer experience (Torous et al., 2021).

In the fiercely competitive real estate market, where user experience and responsiveness are paramount, the implementation of a chatbot emerges as a valuable tool, especially for a business like Fisheye. It not only elevates customer service but also engages potential clients, ultimately driving lead conversion and customer satisfaction.

## **Research Related to Chatbot Development**

An AI-powered chatbot is a promising solution in the real estate sector for delivering personalised help, answering questions, and easing property search processes (Longoni & Cian, 2022). These chatbots are built using advanced algorithms using NLP technologies to analyse data, comprehend the intent of the client to look for multiple property locations, and hence provide relevant responses (Singh, 2023).

[Fisheye](#) provides an enhanced view of the property locations (Fisheye, 2023). This one-of-a-kind visual experience allows prospective buyers to tour properties and obtain a better grasp of the property locations. However, by introducing an AI-powered chatbot, the website of Fisheye Real Estate became more effective for its clients. The chatbot can act as a virtual tour realtor guide, giving on-demand information about property areas and answering user questions.

Productions of the Fisheye Real Estate website rely primarily on visual content. Chatbots can be used with computer vision techniques to provide full assistance (Kumar & Renuka, 2023). These algorithms allow the chatbot to recognise and describe items, extract useful information from photographs or videos, and aid users in identifying specific property traits (Zhao et al., 2022).

The Flask web framework makes it easy to deploy the chatbot as a web application with a unified user experience (Kushwaha & Singh, 2022). Furthermore, the NLTK provides the bot

with linguistic processing skills such as tokenization, stemming, and part-of-speech tagging, which improves its language understanding (Kumar and Renuka, 2023).

Additionally, Keras Libraries in Python provide an easy-to-use API for creating and training neural networks (Chopra & Khurana, 2023). Developers can create a sophisticated chatbot capable of participating in dynamic and contextually-aware discussions with users by combining these technologies. These models allow chatbots to understand better and respond to customer enquiries, improving the user experience in showcasing real estate stock footage images and videos.

Chatbots can use sentiment analysis techniques to provide a more personalised and targeted experience (Chin et al., 2023). By analysing user attitude via text input, the chatbot can tailor its responses, resulting in a more empathic and human-like engagement (Thomas, 2023). Furthermore, including user feedback methods can aid in the long-term improvement of the chatbot's performance (Völkel et al., 2022).

Ensuring the availability and quality of appropriate data is one of the key obstacles in developing an AI-powered chatbot that needs to be addressed (Miljkovic et al., 2023). A vast dataset of real estate-related information, such as property details, market trends, and consumer preferences, is required for training an efficient chatbot (James et al., 2023). It can be difficult to obtain reliable and comprehensive data since it may not be readily available or accessible (McCord, 2022).

Creating an AI-powered chatbot entails dealing with client data such as personal information and questions. Therefore, it is critical to protect user privacy and ensure data security (Miljkovic et al., 2023). Moreover, obtaining user consent for data collecting and following applicable data protection standards are critical ethical issues that need to be addressed while developing an AI-powered chatbot for businesses operating in the real estate industry (Kaur & Solomon, 2022). Hence, Fisheye must put in place strong data security measures, such as encryption, access limits, and secure storage.

It is critical to ensure the correctness and dependability of the chatbot's responses, especially when developing an AI Chatbot for real estate firms where the correctness of location or information regarding land prices carries a huge value (Abioye et al., 2021). In the real estate industry, erroneous or misleading information can have serious ramifications (Treleaven et al., 2021). To ensure that the chatbot's knowledge base represents the most recent information and trends, Fisheye must continue to monitor and upgrade its database information. Regular quality checks and user input can assist in enhancing the chatbot's accuracy and reliability.

Users interacting with the chatbot should be aware of both its abilities and limits (Bauer et al., 2023). When communicating with a chatbot, users should be aware and have access to human aid if necessary (Javed et al., 2023). Being open about the limitations of the chatbot helps to set proper expectations and minimise potential misunderstandings (Zhu, 2023). Therefore, Fisheye must be transparent about the chatbot's AI skills and involvement in decision-making.

The incorporation of an AI-powered chatbot has the potential for Fisheye to revolutionise how clients engage with real estate information. Chatbots can provide personalised advice, facilitate property search operations and boost client engagement by leveraging NLP techniques, computer vision algorithms and sentiment analysis. Addressing ethical concerns and resolving technical hurdles is critical in ensuring the success and widespread adoption of AI-powered chatbots in real estate.

Moreover, Fisheye's AI-powered chatbot should focus on improving client experiences, offering trustworthy information and adhering to ethical standards by tackling the aforementioned challenges. Regular monitoring, continual improvement efforts and user input are required to ensure that the chatbot's performance matches the demands and expectations of the client. To develop the field, future research should focus on improving chatbot capabilities, improving user experience, and investigating new AI technologies.

Previous research has also highlighted the significant role of WordNet and Synsets in enhancing chatbot language understanding. WordNet, as a comprehensive lexical database, aids chatbots in disambiguating word senses and establishing semantic relationships between words (Longo et al., 2023). By incorporating Synsets, chatbots can better navigate contextual nuances, leading to improved response generation (Petralba, 2020). Researchers like Amato, Cozzolino, and Ferraro (2021) have reported that integrating WordNet and Synsets can notably boost a chatbot's ability to discern user intent and provide more contextually relevant answers, resulting in more satisfying user experiences.

Scholars have also recognized the importance of similarity comparison techniques in chatbot responses. These methods enable chatbots to measure the similarity between user input and predefined patterns or knowledge bases (Shashavali et al., 2019). Research has also demonstrated that accurate similarity comparison algorithms, such as cosine similarity or Jaccard similarity, contribute to chatbot responses that are contextually coherent and relevant. They play a vital role in identifying related terms or phrases, thereby enhancing the chatbot's ability to maintain engaging and meaningful conversations (Listl et al., 2021).

The use of tree and treebank structures in chatbot development has gained traction in research (Galitsky and Galitsky, 2021). Linguistic tree structures facilitate syntactic and semantic analysis, allowing chatbots to understand the hierarchical relationships within sentences (Castle-Green et al., 2020). Moreover, Bilgin and Yavuz (2022) have pointed out that integrating tree structures into chatbot architectures improves their ability to parse and generate more natural language responses. This, in turn, leads to chatbots that can engage in conversations with a higher degree of fluency and understanding, making them more effective in various applications.

Named Entity Recognition (NER) has been extensively studied for its contribution to chatbot capabilities, particularly in information extraction and entity identification. Cao and Nguyen (2021) have investigated rule-based and machine learning-based NER techniques, emphasizing their importance in chatbots' ability to recognize and categorize entities within text. NER integration empowers chatbots to provide accurate and contextually relevant information when users inquire about specific entities. This feature enhances the chatbot's utility in domains where precise information retrieval is critical, such as customer support and information search tasks (Zhang, 2020).

## **Challenges Associated with Chatbot Development**

In contrast, developing an AI-powered chatbot for Fisheye comes with several challenges. One of the primary challenges in developing a chatbot for Fisheye is the complexity of the real estate industry itself. Real estate transactions involve a multitude of variables, including property types, locations, prices, legal processes, and customer preferences (Souali et al., 2019). The chatbot needs to be able to understand and process this complex information to provide meaningful responses to users. To address this challenge, NLP algorithms and machine learning models need to be employed to help the chatbot better understand and respond to user queries.

Another significant challenge is ensuring the security and privacy of user data. Real estate transactions often involve sensitive information, such as financial details and personal identification data. To overcome this challenge, stringent data security measures are needed to protect users' confidential information. This includes encrypting data, implementing secure authentication processes, and adhering to relevant data protection regulations, such as the General Data Protection Regulation (GDPR). Therefore, regular security audits and updates are essential to maintaining the chatbot's integrity (Sağlam and Nurse, 2020).

Additionally, developers must verify that AI is used ethically in the design and functionality of the chatbot (Knott, 2023). Avoiding biased or discriminatory behaviours, resolving potential algorithmic biases, and promoting fairness and equal treatment for all clients are all part of this (Zhu & Li, 2022). The chatbot should be built to respect user privacy, offer accurate and fair information, and refrain from using manipulative or misleading tactics (Kaur & Solomon, 2022).

Furthermore, user engagement and satisfaction are pivotal in the real estate industry. Unlike simple customer service chatbots, real estate chatbots must guide users through intricate processes like property searches, mortgage applications, and legal paperwork. Ensuring a seamless user experience is a significant challenge. The chatbot's user interface and conversational design must be intuitive and user-friendly. User feedback and continuous testing can help identify pain points and areas for improvement.

Interoperability is another challenge worth mentioning. Real estate businesses often use multiple software tools and platforms for various tasks, including property management, marketing, and customer relationship management (CRM). The chatbot needs to seamlessly integrate with these systems to provide a holistic user experience. Developing robust APIs and ensuring compatibility with existing software solutions is crucial for overcoming this challenge.

Moreover, maintaining the chatbot's knowledge base is an ongoing challenge. The real estate market is dynamic, with property listings constantly changing, and new market trends emerging. Fisheye Real Estate's chatbot needs to stay up-to-date with the latest market information to provide accurate advice and recommendations to users. Implementing a content management system (CMS) to update property listings and market insights regularly can help address this challenge.

Building trust with users is also essential in the real estate industry. Many users may be making significant financial decisions based on the chatbot's recommendations. Therefore, the chatbot must establish credibility and transparency in its interactions. Providing clear and well-documented information sources, such as property histories and market data, can help build trust. Additionally, offering access to human agents when users need more personalized assistance can enhance trust and satisfaction.

Developing a chatbot that can understand and react to natural language queries is a challenging task. Understanding the intricacies, context, and intent underlying user inquiries can be difficult, particularly in the real estate industry, where questions might be diverse and

domain-specific (Mah et al., 2022). To solve this obstacle, powerful NLP methods and training models on real estate-specific data are required (James et al., 2023). Researchers have also investigated frameworks like Flask, NLTK, and Keras Libraries in Python, which are also effective in creating unique algorithms for chatbots to function effectively (Kushwaha & Singh, 2022; Chopra & Khurana, 2023).

In the next section, a full-scale implementation of the chatbot development is discussed featuring all the discussion on the back-end and front-end development.

## Chapter - 3: Implementation

During the implementation phase of developing a chatbot tailored for Fisheye Real Estate, a meticulous series of steps were undertaken to ensure seamless integration of the technology. This entails delving into the core of Fisheye Real Estate's business objectives and discerning the intricacies of its customer needs. By doing so, the development team gains a profound understanding of the company's aspirations and aligns the chatbot's functionalities accordingly.

Moreover, this phase involves pinpointing the specific tasks that the chatbot executes, such as facilitating property searches and providing informative responses to frequently asked questions. To further refine the implementation strategy, the team meticulously determines the optimal platforms for deploying the chatbot, be it on the company's website or various messaging applications. This holistic approach to requirement analysis lays a robust foundation for the subsequent stages of chatbot development, ensuring a tailor-made solution finely attuned to Fisheye Real Estate's distinct demands.

In the design and architecture phase, the blueprint for the chatbot's functionality and user interactions takes shape, ensuring a user-friendly and efficient experience. This involves meticulously plotting the conversational flow, mapping out how the chatbot engages with users and guiding them through various tasks seamlessly. Concurrently, a critical decision is made regarding the platform or framework for chatbot development. Options like custom development using NLP, NLTK and Keras Libraries in Python were weighed against the project's specific requirements, scalability needs, and ease of integration.

To bolster the chatbot's capabilities, the integration points with Fisheye Real Estate's backend systems and databases were carefully defined. This entails establishing a robust connection between the chatbot and the company's internal infrastructure, enabling real-time access to property listings, user data, and relevant information. The design also incorporates strategies for handling potential data security and privacy concerns, ensuring that customer information remains protected throughout interactions.

In the NLP development phase, the chatbot became capable of comprehending and generating human-like responses. This involved a two-fold process: first, the development of NLP models capable of understanding diverse user inputs, and second, the generation of contextually relevant responses.

The chatbot's NLP models were developed and trained using large datasets to recognize the nuances of user queries. This training equips the chatbot with the linguistic intelligence to accurately interpret user intent, whether they inquire about property listings or seek general information. An important aspect of this development is entity recognition, which enables the chatbot to extract specific details like property features and locations from user messages. This feature enriches user interactions by providing more personalized and targeted responses.

As the chatbot interacts with real users, the NLP models undergo a continuous improvement process. This fine-tuning is essential to adapt the models to real-world language variations, slang, and evolving user behaviours. By analyzing actual user interactions and feedback, the chatbot's accuracy and effectiveness gradually improve, leading to more natural and relevant conversations. This iterative refinement process ensures that the chatbot remains a dynamic and responsive tool for users engaging with Fisheye Real Estate, enhancing customer satisfaction and engagement.

In the backend development phase, the foundational infrastructure was constructed to manage the flow of user requests and responses, forming the backbone of the chatbot's functionality. This involves creating a robust backend system that efficiently handles user interactions while maintaining high performance and reliability.

The integration with Fisheye Real Estate's databases and APIs is a pivotal aspect of this phase. By establishing seamless connections, the chatbot gains access to up-to-date property listings, ensuring that users receive accurate and relevant information. This integration also facilitates the retrieval of other pertinent details, enriching user interactions with comprehensive insights.

User authentication and data security measures were implemented rigorously to safeguard user data and interactions. This involves setting up secure authentication protocols to ensure only authorised users can access sensitive information or perform certain actions. Stringent data security measures, such as encryption and secure communication protocols, were put in place to safeguard user information and maintain privacy.

User Interface (UI) Development played a crucial role in creating an engaging and visually harmonious chatbot experience aligned with Fisheye Real Estate's brand identity. This phase encompassed the design of an intuitive and branded chatbot interface. The UI was crafted to resonate with the company's aesthetics, ensuring a seamless blend between the chatbot and Fisheye Real Estate's visual identity.

During the testing and quality assurance phase, the chatbot's functionality was subjected to rigorous scrutiny across various user scenarios. This includes comprehensive testing to ensure that the chatbot navigates different user inputs accurately, and regression testing was performed to verify that new updates do not disrupt existing features. Any identified bugs, inconsistencies, or usability issues were diligently addressed and rectified, ensuring that the chatbot delivers a flawless user experience.

The final deployment phase involved launching the chatbot onto the designated platforms, such as the Fisheye Real Estate website and messaging apps. Thorough compatibility and integration checks were carried out to guarantee seamless performance across diverse devices and browsers. This meticulous attention to detail ensures that users can access the chatbot effortlessly, regardless of their chosen platform, fostering widespread engagement and satisfaction.

The training and optimization phase is an ongoing process crucial for maintaining the chatbot's effectiveness and responsiveness. Continuous monitoring of the chatbot's performance and user interactions is essential to gauge its efficiency and identify areas for improvement. This involves closely tracking metrics such as user satisfaction, response accuracy, and engagement rates.

User feedback became a valuable resource during this phase. By actively collecting and analyzing feedback, insights were collected regarding user preferences, pain points, and areas where the chatbot might fall short. This feedback was used to refine the chatbot's responses, behaviour, and conversational flow, resulting in a more user-centric and effective interaction.

The maintenance and updates phase proved to be a pivotal aspect of ensuring the chatbot's sustained performance and relevance over time. Ongoing maintenance and support were provided to promptly address any issues that might arise and to make necessary changes as circumstances evolve. This proactive approach made sure that the chatbot remained a dependable and functional tool for users engaging with Fisheye Real Estate.

Regular updates also played a vital role in keeping the chatbot current and valuable. New property listings, features, and improvements were incorporated based on user feedback and market trends. This iterative process involved staying attuned to user preferences and behaviours, as well as adapting to shifts in the real estate landscape. By consistently enhancing the chatbot's capabilities, it remained a valuable asset that provided accurate information and a seamless experience to users.

Incorporating analytics and reporting mechanisms was integral to measuring the effectiveness and impact of the chatbot on Fisheye Real Estate's operations. This phase involved the implementation of robust analytics tools to track key performance indicators (KPIs) such as user engagement, conversion rates, user satisfaction, and more. These metrics provided valuable insights into how users are interacting with the chatbot and the outcomes of those interactions.

Reports were generated based on the collected data to comprehensively assess the chatbot's contribution to Fisheye Real Estate's business goals. These reports offer a clear picture of the chatbot's performance, highlighting areas of success and areas that may require further improvement. By leveraging these insights, data-driven decisions enhanced the chatbot's functionality, addressed user pain points, and aligned more closely with the company's objectives.

## **WordNet and Synsets**

As discussed in the literature review, WordNet is a comprehensive lexical database that categorizes words into groups of synonyms known as synsets and establishes semantic relationships between words. Leveraging WordNet in chatbot development significantly enhanced its understanding and response generation capabilities. For instance, when a user typed "house" in their query, the chatbot did not only recognize this keyword but also uncovered associated terms such as "home," "property," and "residence." This functionality empowered the chatbot to offer responses that are not only linguistically precise but also aligned with the user's intent. By employing WordNet's insights, the chatbot delivered contextually rich answers that cater to the nuanced vocabulary of the real estate domain.

## **Similarity Comparison**

Implementing similarity comparison within the chatbot significantly enhanced its ability to comprehend user intent and deliver accurate responses. This approach quantified the likeness between user input and existing data to pinpoint the most relevant answer. The chatbot effectively gauged what the user was seeking by contrasting user queries with an extensive database of real estate listings, FAQs, and pertinent content. Techniques such as cosine similarity or word embedding-based similarity played a pivotal role in this process. These techniques mathematically evaluated the proximity of words and phrases, allowing the chatbot to identify suitable responses even when the wording varies. Consequently, users received tailored and spot-on information that catered precisely to their queries, heightening the overall user experience.

## **Tree and Treebank**

Within the realm of NLP, a tree structure, often referred to as a parse tree, serves as a graphical representation of a sentence's grammatical arrangement. This hierarchical depiction captures the relationships between words and phrases, showcasing how they function in the sentence's syntax. Accompanying this is the concept of a treebank, a compilation of sentences that have been annotated with their respective parse trees. By employing these notions, the proposed chatbot delved into the grammatical makeup of user queries to ensure a precise comprehension of their intent.

For instance, when a user submitted a query that involved both looking for pictures of house and studio apartments in Miami, the chatbot discerned the underlying grammatical structure and identified the nuances within the sentence through the analysis of the parse tree. This sophisticated understanding enabled the chatbot to distinguish between the user's interest in pictures and videos of both house and studio apartments. Consequently, the chatbot delivered responses that were exceptionally pertinent to the user's specific inquiries. By leveraging the power of parse trees and treebanks, the chatbot elevated its conversational capabilities to an advanced level, offering users tailored and contextually precise information.

## **Named Entity Recognition (NER)**

NER is a pivotal process within NLP that entails the detection and categorization of named entities within the text, encompassing entities like names of individuals, locations, dates, and more. For the chatbot, NER emerged as a potent tool for extracting crucial details from user input, thereby facilitating a comprehensive understanding of their requirements.

So when a user submitted a query that reads, "I'm looking for drone footage of Empire State Building in New York", employing NER, the chatbot discerned and labelled essential pieces of information, such as "drone footage," "Empire State Building," and "New York". This granular analysis enabled the chatbot to gauge the user's preferences accurately.

Subsequently, the chatbot tailored its responses to provide pertinent options that align with the user's desires. For instance, it furnished pictures and video listings that matched the specified location. This precision-driven approach amplified the user experience by presenting information that is not only relevant but also aligned with their specific needs.

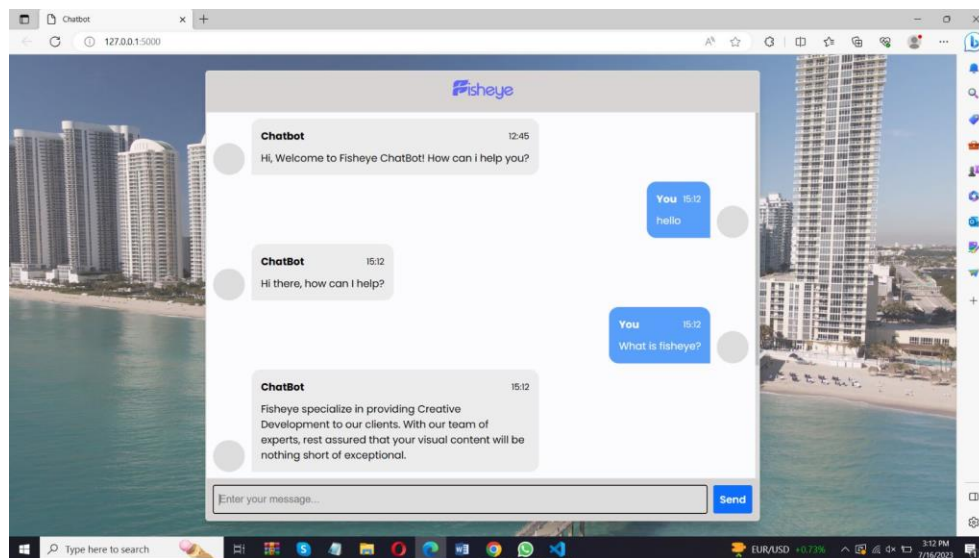
By harnessing the capabilities of Named Entity Recognition, the chatbot served as a knowledgeable assistant, effortlessly discerning user preferences and furnishing responses that are attuned to the precise details of their property footage search.



## Chapter - 4: Project Work Evaluation Strategy / Test Plan

As discussed in the literature review, chatbot development involves NLP to understand and interpret user input. It involves techniques like text segmentation, part-of-speech tagging, named entity recognition, and sentiment analysis. NLP helps the chatbot understand user queries and extract relevant information. Moreover, machine learning algorithms including decision trees, support vector machines (SVM), and random forests were used during the development stages.

Additionally, intent recognition techniques like supervised learning involving labelled examples of user queries and their corresponding intents were used to train the chatbot. Dialogue management techniques, such as Markov Decision Processes (MDPs) or Deep Q-Networks (DQNs), were employed to manage the dialogue and select the most suitable response based on the current conversation context. Most importantly, database management systems or information retrieval techniques were used to equip the chatbot to store and retrieve information efficiently. This allows the chatbot to provide accurate and up-to-date responses to client queries.



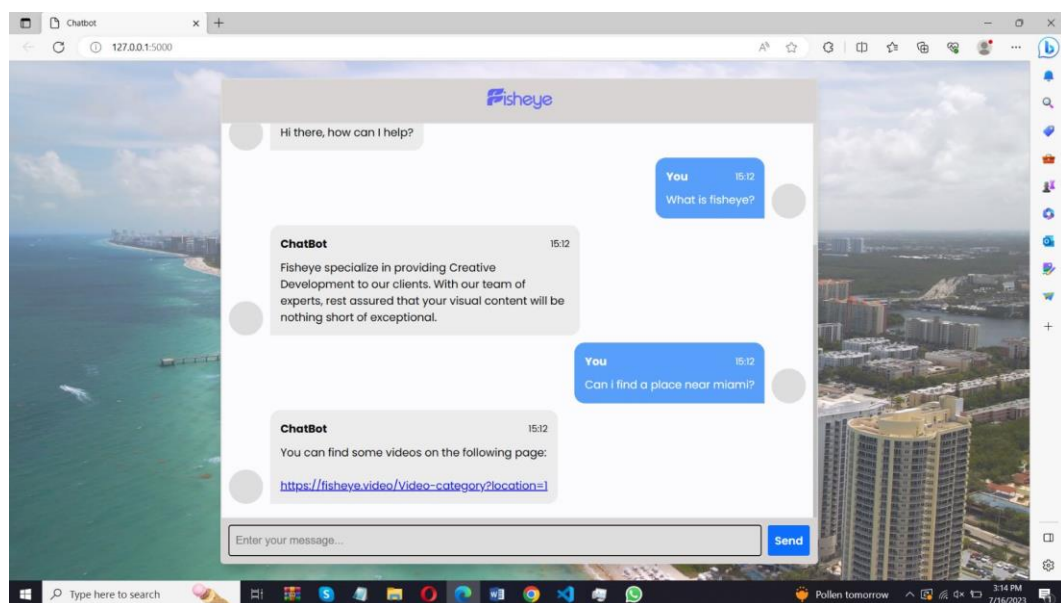
*Fig 4.1: An overview of an ongoing chat between the user and the chatbot*

As the chatbot is programmed to initiate the conversation with the client, it drops a message as soon as the customer arrives at the desired landing page of the website. The standard response embedded into the chatbot to start the conversation is “**Hi, Welcome to the Fisheye Chatbot! How can I help you?**” This message is designed to build intrigue so that the client feels comfortable asking questions from the chatbot just like they might do while interacting

with a real human. In this way, a conversation develops with an effort to satisfy the clients with their queries. It also tells them that it's a chatbot and not a person.

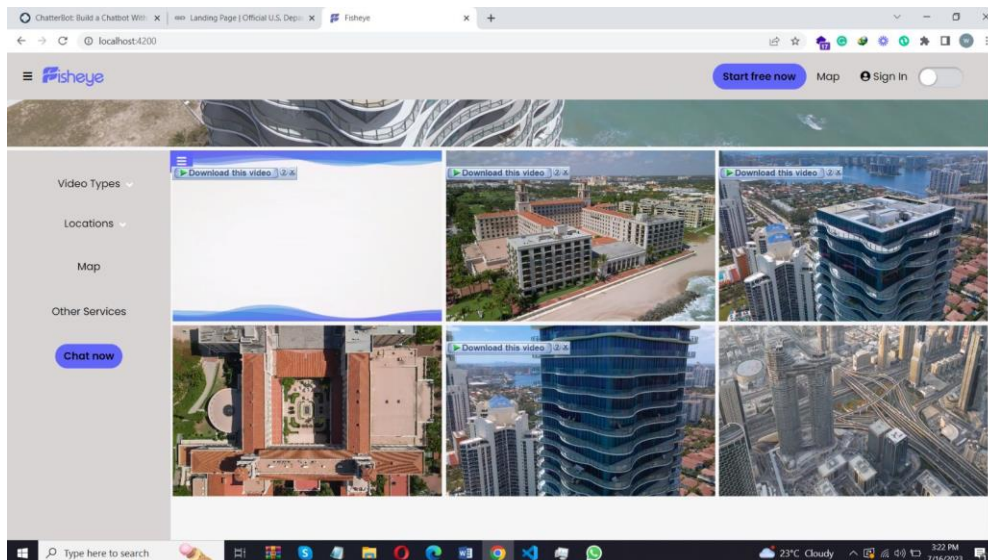
The chatbot recognizes client intents and understands natural language. Therefore, other than greeting and introduction, the clients can also ask questions or submit information about their preferences, such as the type of property, location, budget, or any other real estate-related enquiries. Moreover, clients can instruct the chatbot to look for suitable properties depending on their specifications. The chatbot queries its database and returns a list of appropriate property listings, including price, size, location, and photos.

Additionally, the chatbot answers frequently asked questions (FAQs) concerning Fisheye Real Estate's services and policies, as well as general real estate inquiries. If it is unable to answer a question, it can redirect the user to a useful link for further information.



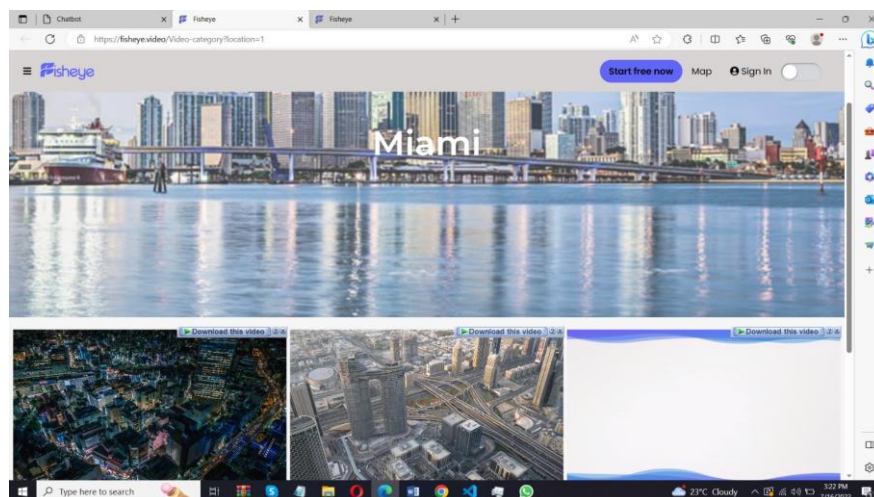
*Fig 4.2: A further overview of an ongoing chat between the user and chatbot*

The above image shows how the chatbot interacts with the potential client. Once the client starts an interaction with the chatbot, it quickly makes a reply to ensure the client proceeds with his or her query. For example, if a client asks the question “What is Fisheye?” the chatbot responds with a short introduction to the stock footage company. Moreover, if the client asks about property locations around Miami, the chatbot responds with a link that redirects the client towards property locations in and around Miami.



*Fig 4.3: An overview of the homepage of the Fisheye website from where the chatbot is accessible*

The above image displays a layout of the Fisheye website where the chatbot is integrated. The chatbot pops up in the bottom right corner once the user clicks on the “chat now” button. Therefore, it is convenient for the user to start communication with the chatbot.



*Fig 4.4: An overview of the main landing page of the Fisheye website*

This above snapshot is the main landing page of the website where the chatbot primarily appears and interacts with the potential client.

## App.py

```
1 # app.py
2 import nltk
3 nltk.download('popular')
4 from nltk.stem import WordNetLemmatizer
5 lemmatizer = WordNetLemmatizer()
6 import pickle
7 import numpy as np
8 import requests
9
10 from keras.models import load_model
11 model = load_model('model.h5')
12
13 import json
14 import random
15 intents = json.loads(open('data.json').read())
16 type_response_API = requests.get('https://content.fisheye.video/api/video/types')
17 type_data_file = type_response_API.text
18 location_response_API = requests.get('https://content.fisheye.video/api/locations')
19 locations_data_file = location_response_API.text
20 locations = json.loads(locations_data_file)
21 type = json.loads(type_data_file)
22 words = pickle.load(open('words.pkl', 'rb'))
23 classes = pickle.load(open('labels.pkl', 'rb'))
24
25 def clean_up_sentence(sentence):
26     # tokenize the pattern - split words into array
27     sentence_words = nltk.word_tokenize(sentence)
28     # stem each word - create short form for word
29     sentence_words = [lemmatizer.lemmatize(word.lower()) for word in sentence_words]
30     return sentence_words
31
32 # return bag of words array: 0 or 1 for each word in the bag that exists in the sentence
33
34 def bow(sentence, words, show_details=True):
35     # tokenize the pattern
36     sentence_words = clean_up_sentence(sentence)
37     # bag of words - matrix of N words, vocabulary matrix
38     bag = [0]*len(words)
39     for s in sentence_words:
40         for i,w in enumerate(words):
41             if w == s:
42                 # assign 1 if current word is in the vocabulary position
43                 bag[i] = 1
44                 if show_details:
45                     print("found in bag: %s" % w)
46     return np.array(bag)
```

Fig 4.5: An overview of App.py back-end coding

The coding snapshots displayed in this section show the working of App.py. Here App.py is breaking data coming down from API to ensure that the data becomes understandable for the system. The app.py file, as shown above, uses Flask to establish a web server to handle incoming HTTP POST requests. The chatbot's endpoint is /webhook, where it listens for user messages. The webhook function is activated when a message is received.

The process\_user\_input function is in charge of accepting user input, determining the user's purpose, and creating a suitable answer. This function would require more complex NLP and possibly communicating with external APIs to obtain property data and offer relevant information in the case of a real chatbot.

```
1 # app.py
2
3 def predict_class(sentence, model):
4     # filter out predictions below a threshold
5     p = bow(sentence, words, show_details=False)
6     res = model.predict(np.array([p]))[0]
7     ERROR_THRESHOLD = 0.25
8     results = [(i,r) for i,r in enumerate(res) if r>ERROR_THRESHOLD]
9     # sort by strength of probability
10    results.sort(key=lambda x: x[1], reverse=True)
11    return_list = []
12    for r in results:
13        return_list.append(("intent": classes[r[0]], "probability": str(r[1])))
14    return return_list
15
16 def getResponse(ints, intents_json, locations_json, type_json):
17    tag = ints[0]['intent']
18    list_of_intents = intents_json['intents']
19    list_of_locations = locations_json['data']
20    list_of_type = type_json['data']
21    for i in list_of_intents:
22        if(i['tag'] == tag):
23            result = random.choice(i['responses'])
24            break
25    for j in list_of_locations:
26        if(j['name'] == tag):
27            result = str(j['id'])
28            result = "<p>You can find some videos on the following page:</p><a target='_blank' href='\""+https://fisheye.video/Video-category/location="+result+"\"'+>https://fisheye.video/Video-category/location="+result+"</a>"
29            break
30    for k in list_of_type:
31        if(k['name'] == tag):
32            result = str(k['id'])
33            result = "<p>You can find relevant videos on the following page:</p><a target='_blank' href='\""+https://fisheye.video/Video-category/type="+result+"\"'+>https://fisheye.video/Video-category/type="+result+"</a>"
34            break
35    return result
36
37 def chatbot_response(msg):
38    ints = predict_class(msg, model)
39    res = getResponse(ints, intents_json, locations_json, type_json)
40    return res
41
42 from flask import Flask, render_template, request
43
44 app = Flask(__name__)
45 app.static_folder = 'static'
```

Fig 4.6: A further overview of App.py back-end coding

For example, if a potential client comes on the landing page of the website and starts an interaction with the chatbot by typing “Hi”, the chatbot analyses and checks the response to the greetings like “Hi” in the back end and answers back to the user accordingly. Moving ahead, if the user types in a query saying “Find me a place near Miami”, the chatbot is triggered with the keyword here which is “Miami”. So in response, the chatbot searches the data in API in the form of JSON files and previews the results relevant to the keyword.

## Flask Code for Running and Rendering HTML Data

To combine the Flask code with HTML to create a functional Fisheye Real Estate chatbot, we'll first create a basic user interface where clients may enter their queries and receive the chatbot's responses. The Flask code handles incoming requests, processes user input, and returns appropriate replies for display on the HTML page.

Here is how the basic user interface was created:

1. **Create an HTML template:** An HTML file (e.g., index.html) is created that acts as the chatbot's user interface, with a form where users can submit their messages and an area to show the chatbot's responses.
2. **Update app.py:** Changes were made to the app.py file to generate the HTML template and process user input from the chat interface.
3. **Start the application:** Both the app.py and index.html files should be saved in the same directory. Navigate to the directory and execute the Flask application.
4. **Interact with the Chatbot:** The chatbot processes client input and responds with relevant information based on implementing the process\_user\_input function.

```
80 def chatbot_response(msg):
81     ints = predict_class(msg, model)
82     res = getResponse(ints, intents, locations, type)
83     return res
84
85
86 from flask import Flask, render_template, request
87
88 app = Flask(__name__)
89 app.static_folder = 'static'
90
91 @app.route("/")
92 def home():
93     return render_template("Chatbot.html")
94
95 @app.route("/get")
96 def get_bot_response():
97     userText = request.args.get('msg')
98     return chatbot_response(userText)
99
100
101 if __name__ == "__main__":
102     app.run()
```

*Fig 4.7: An overview of Flask backend coding*

The above snapshot indicates the code development for the chatbot using the Flask framework used to execute Python HTML coding. The framework is helpful because HTML coding cannot be executed directly. Hence, the code is rendered with a combination of Flask and App.py where the latter is utilizing Flask to target the HTML file and run the code.

## **Training the File for Chatbot Understanding - Training.py**

Training.py is an essential component of chatbot development where the chatbot learns to interpret data stored in JSON files by learning different responses to user inputs. Here's a general outline of how training.py works for training the chatbot. Firstly, relevant data was collected for training the chatbot. This data could include conversations related to real estate, user queries, and corresponding responses from real estate experts or historical chat logs. The collected data was cleaned and preprocessed through tokenization, removing stop words, stemming or lemmatization, and converting the text data into a format suitable for machine learning models.

Moreover, the chatbot was trained to determine user intent. This can be done using sequence-to-sequence models, where the input sequence (user query) is mapped to an output sequence (chatbot response). For example, the chatbot classifies whether a user is looking for property listings, asking about prices, seeking agent assistance, or any other real estate-related intent. Additionally, through Named Entity Recognition (NER), the chatbot identifies important entities within user queries, such as property location, budget, property type, etc. NER helps extract crucial information to process user requests effectively.

The performance of the trained chatbot is evaluated using metrics like accuracy or custom evaluation methods to ensure it meets the desired level of performance. The trained model was integrated with the Flask-based chat interface as shown previously. This allows users to interact with the chatbot through the web interface and receive responses based on the trained model's output.

Finally, continuous learning and improvement mechanisms were implemented in the chatbot development process. This can involve collecting client feedback, updating the training data, and periodically retraining the model to keep it up-to-date with changing real estate trends and user preferences.

The following coding screen displays the development process of training.py to ensure that the chatbot is fully capable of responding to all types of queries:

```

1 import nltk
2 from nltk.stem import WordNetLemmatizer
3 lemmatizer = WordNetLemmatizer()
4 import json
5 import pickle
6 import requests
7
8 import numpy as np
9 from keras.models import Sequential
10 from keras.layers import Dense, Activation, Dropout
11 from keras.optimizers import SGD
12 import random
13
14 words = []
15 more_new_words = []
16 new_words = []
17 classes = []
18 documents = []
19 ignore_words = ['?', '!', '']
20 data_file = open('data.json').read()
21 intents = json.loads(data_file)
22 response_API = requests.get('https://content.fisheye.video/api/locations')
23 location_data_file = response_API.json()
24 type_response_API = requests.get('https://content.fisheye.video/api/video_types')
25 type_data_file = type_response_API.json()
26
27 for location in location_data_file['data']:
28     #tokenize each word
29     x = nltk.word_tokenize(location['name'])
30     new_words.extend(x)
31     print('locations: ' + x)
32     #add documents in the corpus
33     documents.append((x, location['name']))
34
35     # add to our classes list
36     if location['name'] not in classes:
37         classes.append(location['name'])
38
39 for type in type_data_file['data']:
40     #tokenize each word
41     y = nltk.word_tokenize(type['name'])
42     more_new_words.extend(y)
43     print('types: ' + y)

```

Fig 4.8: An overview of Training.py back-end coding

For example, the chatbot trains itself to break the code instructions and learn to generate responses to various inputs such as greetings including “Hi” or “Hello” and gratitude words like “Thank You”.

```

82 pickle.dump(words, open('texts.pkl', 'wb'))
83 pickle.dump(classes, open('labels.pkl', 'wb'))
84
85 # create our training data
86 training = []
87 # create an empty array for our output
88 output_empty = [0] * len(classes)
89 # training set, bag of words for each sentence
90 for doc in documents:
91     # initialize our bag of words
92     bag = []
93     # list of tokenized words for the pattern
94     pattern_words = doc[0]
95     # lemmatize each word - create base word, in attempt to represent related words
96     pattern_words = [lemmatizer.lemmatize(word.lower()) for word in pattern_words]
97     # create our bag of words array with 1, if word match found in current pattern
98     for w in words:
99         bag.append(1) if w in pattern_words else bag.append(0)
100
101     # output is a '0' for each tag and '1' for current tag (for each pattern)
102     output_row = list(output_empty)
103     output_row[classes.index(doc[1])] = 1
104
105     training.append([bag, output_row])
106 # shuffle our features and turn into np.array
107 random.shuffle(training)
108 training = np.array(training)
109 # create train and test lists. X - patterns, Y - intents
110 train_x = list(training[:,0])
111 train_y = list(training[:,1])
112 print("Training data created")
113
114 # Create model - 3 layers. First layer 128 neurons, second layer 64 neurons and 3rd output layer contains number of neurons
115 # equal to number of intents to predict output intent with softmax
116 model = Sequential()
117 model.add(Dense(128, input_shape=(len(train_x[0]),), activation='relu'))
118 model.add(Dropout(0.5))
119 model.add(Dense(64, activation='relu'))
120 model.add(Dropout(0.5))
121 model.add(Dense(len(train_y[0]), activation='softmax'))
122
123 # Compile model. Stochastic gradient descent with Nesterov accelerated gradient gives good results for this model
124 sgd = SGD(lr=0.01, decay=1e-6, momentum=0.9, nesterov=True)
125 model.compile(loss='categorical_crossentropy', optimizer=sgd, metrics=['accuracy'])

```

Fig 4.9: A further overview of Training.py back-end coding

After the completion of training, the model was created which is shown in the coding as follows:

```

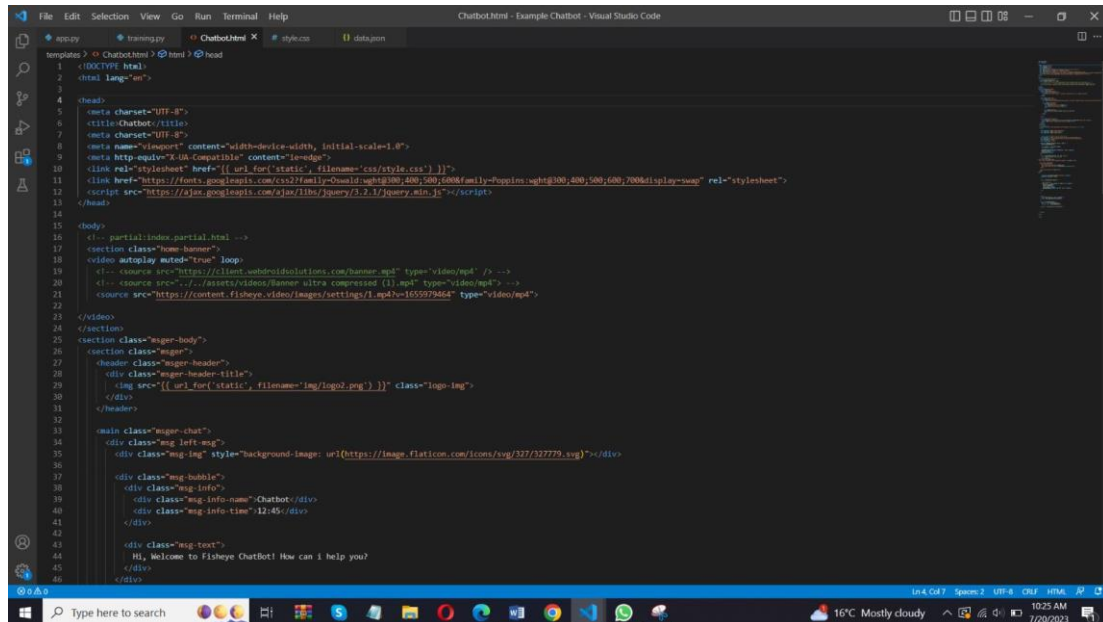
128 #fitting and saving the model
129 hist = model.fit(np.array(train_x), np.array(train_y), epochs=200, batch_size=5, verbose=1)
130 model.save('model.h5', hist)
131
132 print("model created")

```

Fig 4.10: An overview of model coding after the completion of Training

## Chatbot HTML Code

The development process made sure that the data is sent in the form of an HTML file to App.py which then checks whether the requested data is available in Training.py or not. It means that if the Chatbot can find the data in the API, it responds to client queries by sending the relevant links that guide the user on its queries.

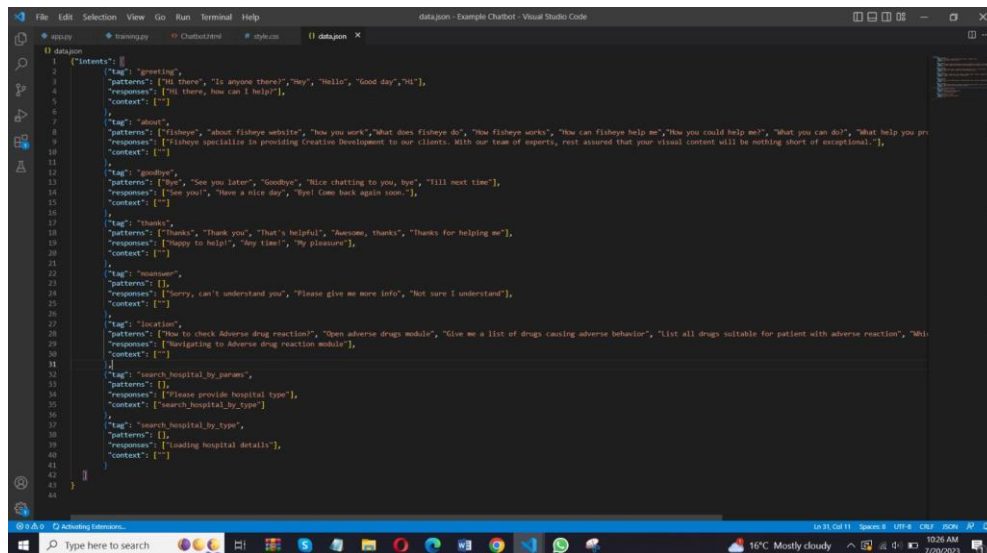


```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <meta charset="UTF-8">
6   <title>Chatbot</title>
7   <meta charset="UTF-8">
8   <meta name="viewport" content="width=device-width, initial-scale=1.0">
9   <meta http-equiv="X-UA-Compatible" content="ie=edge">
10  <link rel="stylesheet" href="{url_for('static', filename='css/style.css')}">
11  <link href="https://font.googleapis.com/css?family=Charm:400,480,500,600,700&display=swap" rel="stylesheet">
12  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
13 </head>
14
15 <body>
16   <!-- partial:index.partial.html -->
17   <section class="home-banner">
18     <video autoplay muted="True" loop>
19       <!-- source src="https://client.webdroidolutions.com/banner.mp4" type="video/mp4" /> -->
20       <!-- source src="{url_for('static', filename='assets/videos/Banner ultra compressed (1).mp4" type="video/mp4" /> -->
21       <source src="https://content.fisheye.video/images/settings/1.mp4?v=105527946d" type="video/mp4">
22     </video>
23   </section>
24   <section class="muger-body">
25     <section class="muger">
26       <header class="muger-header">
27         <div class="muger-header-title">
28           
29         </div>
30       </header>
31     </section>
32   </section>
33   <main class="muger-chat">
34     <div class="mug-left-mug">
35       <div class="mug-img" style="background-image: url(https://image.flaticon.com/icons/svg/327/327779.svg);">
36     </div>
37     <div class="mug-bubble">
38       <div class="mug-info">
39         <div class="mug-info-name">Chatbot</div>
40         <div class="mug-info-time">12:45</div>
41       </div>
42       <div class="mug-text">
43         Hi, Welcome to Fisheye Chatbot! How can i help you?
44       </div>
45     </div>
46   </main>
```

Fig 4.11: An Overview of Integration of HTML Code with App.py and API

## Data File

The data file has various responses stored in the form of JSON files where the chatbot answers the client as per their requirement. As discussed in the App.py and Training.py sub-sections, this data is used by the chatbot to fetch responses. If it cannot find any relevant answer, it informs the user about it. Following is a glimpse of the data file made during the coding process.



*Fig 4.12: An overview of Data Files stored in the form of JSON Files*

In the next section, there will be a detailed discussion on the technical documentation and user documentation to get a walkthrough of back-end development and front-end interface overview of the chatbot.

## Chapter - 5: Documentation

In the documentation phase, it becomes evident that the success of the chatbot hinges on crafting both technical and user-oriented documentation. The technical documentation encompasses essential elements such as the intricacies of the system architecture, detailing how the chatbot's components harmonize to facilitate seamless interactions. It further expounds on the APIs and integrations employed to gather external data, shedding light on the mechanics behind the chatbot's intelligence.

The documentation extends to elucidating the data flow within the system, unveiling the journey of user queries and responses. Moreover, the chatbot's decision-making process, or logic, is meticulously outlined, offering insights into the intricate mechanisms governing the chatbot's adept responses. This comprehensive technical documentation lays the groundwork for efficient management and maintenance of the chatbot system, ensuring that it operates optimally and aligns with Fisheye Real Estate's goals.

While technical documentation entails explaining the intricacies of the system architecture, user documentation delves into the flow of data within the system, uncovering the journey of user interactions from queries to responses. The underlying chatbot logic is also expounded, providing insight into the decision-making process governing its intelligent responses.

Conversely, user documentation serves to acquaint users with the chatbot's capabilities. The guide outlines how users can initiate interactions, emphasizes the array of features at their disposal, and elucidates the commands and syntax recognized by the chatbot. Troubleshooting strategies address common user issues, while best practices optimize user experiences. This comprehensive documentation symbiosis proves vital for the streamlined development and successful deployment of the Fisheye Real Estate chatbot.

### Technical Documentation

Several key components and scripts played a pivotal role in bringing the chatbot to life. This section delves into the technical documentation of the programming codes that made up the core of the chatbot's functionality in which there will be a specific focus on four critical files: ``App.py``, ``Training.py``, ``Python.py``, and ``Test.py`` (detailed coding of these files is available in the appendix section), each serving a distinct purpose in the chatbot's architecture.

`App.py` was the entry point of the chatbot application. In this file, the chatbot's interface defined the conversation flow and established connections with various APIs and databases. It was

responsible for handling user inputs, processing them, and generating responses, essentially acting as the bridge between the user and the chatbot's underlying logic.

Training.py implemented the training pipeline for the chatbot's natural language understanding model. They used machine learning frameworks and Keras libraries to train the model on a diverse dataset of real estate-related conversations. This rigorous training ensured that the chatbot could understand and respond accurately to user queries.

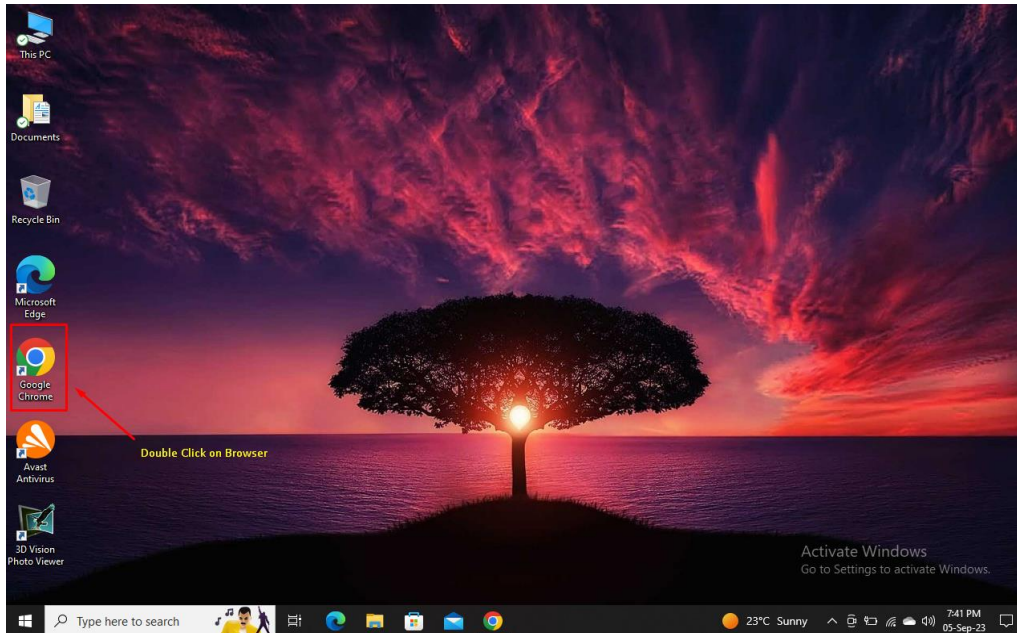
Python.py accommodated the core logic of the chatbot. This file included functions and classes responsible for parsing user input, generating responses, and handling various real estate stock footage data, such as property visuals, footage, and location details. It served as the brain of the chatbot, orchestrating its responses and actions.

Quality assurance was a critical aspect of chatbot development, and Test.py was pivotal in this regard. A suite of test cases was created within this file to evaluate the chatbot's performance. This stimulated user interactions and rigorously verified that the chatbot responded correctly and consistently. This file played an essential role in identifying and addressing any issues or bugs in the chatbot's functionality.

Collectively, the aforementioned four files enabled the chatbot to provide accurate and helpful responses to user inquiries related to real estate footage, establishing it as a valuable tool for users seeking information and assistance in the real estate domain. The careful design and development of these scripts ensured that the chatbot delivered a seamless and user-friendly experience.

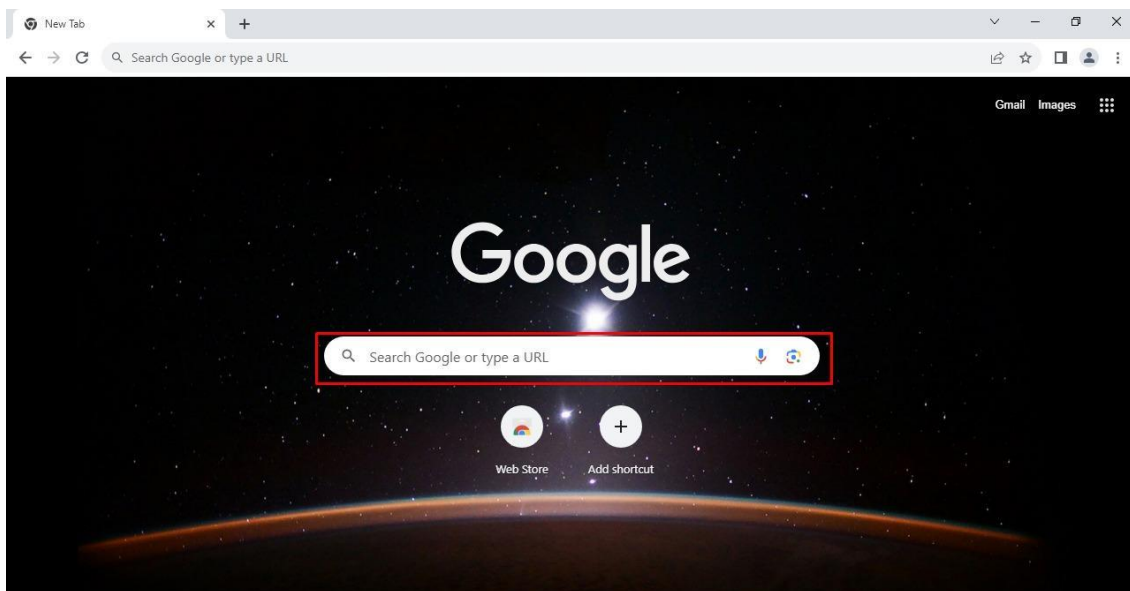
## **User Documentation - How to Run The Chatbot**

As a user, the chatbot can be accessed by visiting [www.fisheye.video](http://www.fisheye.video) on the browser. The following snapshots will give a walkthrough of how the user will eventually land on the aforementioned website:



*Fig 5.1: An overview of the Front-end Desktop view from where the user can access the Fisheye website by clicking on the browser*

After clicking on the browser, the following interface will appear:



*Fig 5.2: A browser overview from where the user can access the Fisheye website*

Here, the user will type in [www.fisheye.video](http://www.fisheye.video), after which the browser will redirect the user to the homepage of the Fisheye website which is shown as follows:

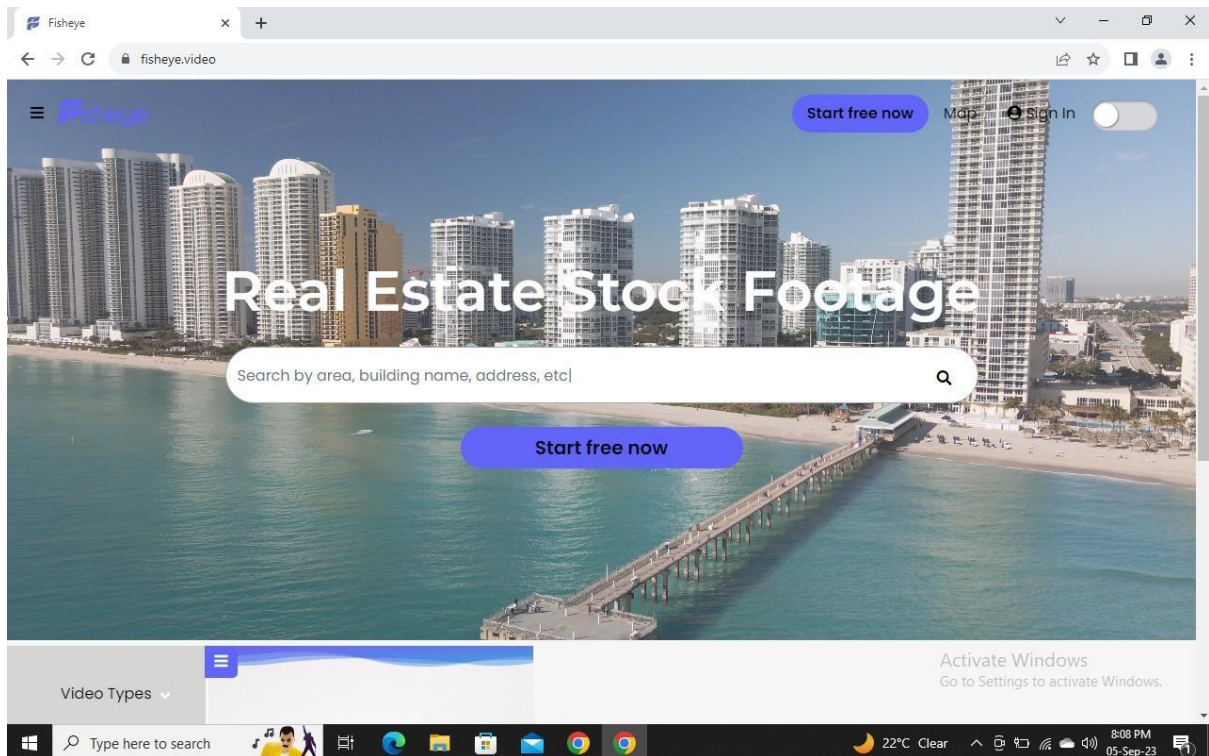


Fig 5.3: An overview of the website interface once the user enters [www.fisheye.video](http://www.fisheye.video) in the search bar

Once the users scroll the homepage, they will find the “Chat Now” button that is embedded with the chatbot:

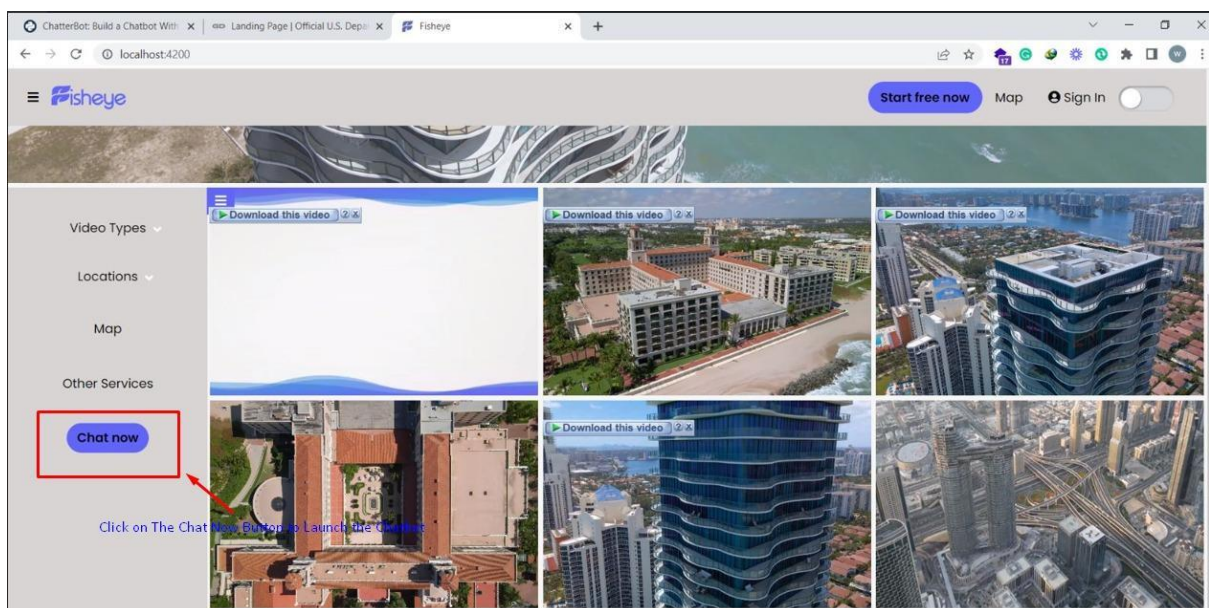
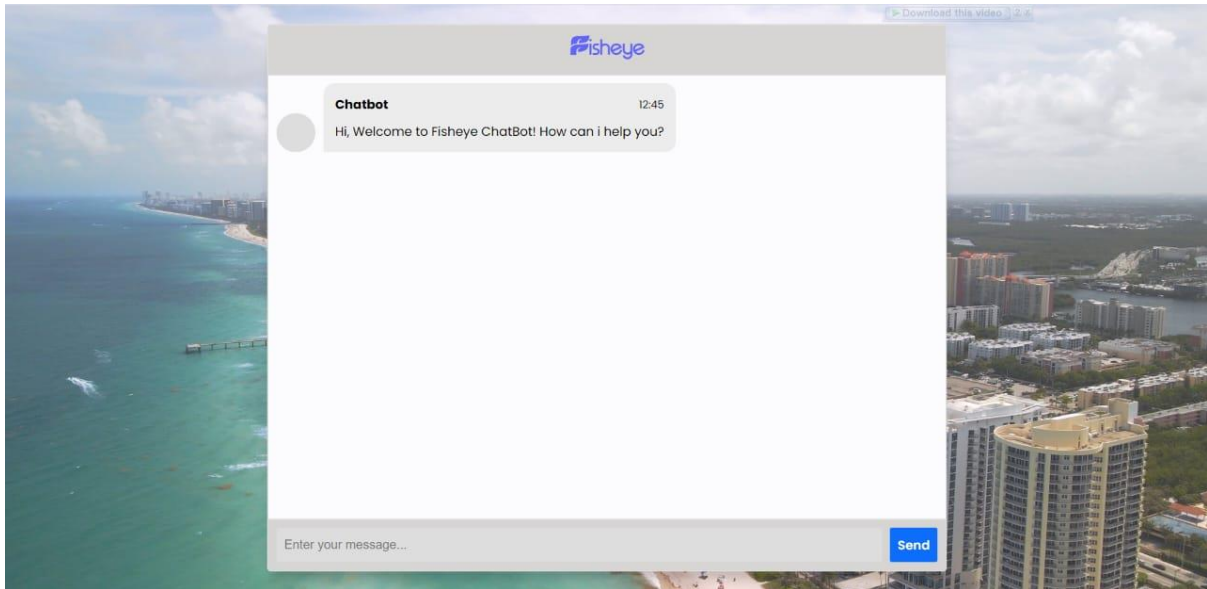


Fig 5.4: The “Chat Now” box is highlighted in Red from where the user can start its interaction with the chatbot

After clicking the “Chat Now” button, a chatbot will pop up in front of the user as shown below:



*Fig 5.5: The first message of the chatbot from where the conversation begins*

The chatbot will start its interaction with the user at first and will continue to respond to user messages after every input. It will guide users through various stages of the real estate process, from initial property searches to mortgage inquiries. It will maintain a friendly and professional tone throughout the interaction, ensuring that users feel comfortable and well-assisted.

Once the user session ends with the chatbot, the user will be redirected towards the following feedback form which is linked by Survey Monkey.

## Fisheye Chatbook Feedback

### Fisheye Chatbot

\* 1. While interacting with the chatbot was your query resolved?

☐ Yes

☐ No

\* 2. Were you satisfied while interacting with the chatbot?

☐ Yes

☐ No

3. What are the issues that you find while interacting with the chatbot and what improvements would you like to have?

\* 4. On a scale of 1 - 5, how satisfied you were with the response of the chatbot?  
1 being extremely dissatisfied and 5 being extremely Satisfied.

☐ 1

☐ 2

☐ 3

☐ 4

☐ 5

Done

Powered by  
 SurveyMonkey  
[See how easy it is to create a survey.](#)

*Fig 5.6: An Overview of Survey Monkey Feedback Form Integrated with the Chatbot*

Using the above form, the users provided feedback on whether the chatbot was able to resolve their query. They gave their opinion on whether they were satisfied with their interaction with the chatbot or not. The feedback form also asked for user insight on how the chatbot can be further improved and those suggested improvements can be integrated. The last question quantified the response of the user on a scale of 1 to 5 which can provide a clear indicator of how much satisfied or unsatisfied the user was with their chatbot interaction.

In the next section, the project will be evaluated based on performance metrics and merits & demerits.

## **Chapter - 6: Evaluation**

The evaluation section begins with an overview of the project, summarizing all the stages of chatbot development and evaluating its usefulness for the designated stock footage company - Fisheye. Moreover, the performance and effectiveness of the chatbot are assessed by measuring performance metrics, tracking user satisfaction, evaluating task completion rates, assessing conversion rates, examining integration with support channels, and verifying the accuracy of property information. Through this comprehensive evaluation process, the chatbot's past performance was thoroughly analyzed to ensure its alignment with intended goals and to drive continuous enhancements for an improved user experience.

### **Overview of the Project**

The project of developing an AI-powered chatbot for Fisheye Real Estate Stock Footage Productions represents a significant step towards enhancing customer engagement, operational efficiency, and data-driven decision-making within the company. This endeavour aims to leverage the capabilities of artificial intelligence to provide real-time, intelligent assistance to clients, prospects, and internal teams in the dynamic realm of real estate.

This project focused on all key development stages after which the chatbot came into existence. After analysing the usefulness of the chatbot in the spotlight of the literature review, the implementation phase was thoroughly discussed. The implementation stage featured essential components of front-end and back-end development work which reinforced the chatbot to become fully functional for public use. Various test strategies were applied to the developed chatbot to identify potential errors and they were eventually eliminated before the documentation phase. Now, the chatbot has been delivered to the client who is satisfied and ready to integrate it with the live web view of the Fisheye website.

### **Performance Metrics**

Performance metrics were employed to gauge various aspects of the chatbot's functionality. Firstly, response time was measured across different query types, providing insight into the chatbot's efficiency in delivering prompt replies. The accuracy of responses and the chatbot's ability to comprehend user intent were also monitored to ensure quality interactions. This metric was utilised while assessing the survey monkey feedback provided by various users in which the users highlighted that the time taken by chatbot to respond to queries was highly efficient.

Moreover, user satisfaction played a pivotal role in evaluating the chatbot's success. This was achieved through user surveys and feedback sessions, which provided valuable input on the users' overall experience. Additionally, analyzing user sentiment derived from their interactions further informed the chatbot's performance. Again, Survey Monkey feedback was also assessed on the parameters of user satisfaction on the basis of which it was concluded that the majority of users were satisfied with the chatbot's performance.

The task completion rate showcased how effectively the chatbot aided users in achieving their objectives, such as property searches or appointment scheduling. Moreover, conversion rates were assessed to determine whether the chatbot contributed to increased lead generation, property inquiries, and sales.

The chatbot's integration with support channels was a crucial assessment point. Its ability to divert support requests from other channels like phone and email, thereby enhancing customer service efficiency, was closely examined.

Lastly, the accuracy of the property information provided by the chatbot was verified to ensure users received up-to-date and precise property listings and details. Overall, this comprehensive evaluation process ensured the chatbot's alignment with its intended goals and its continual enhancement to offer a seamless and valuable user experience.

## **Advantages of The Chatbot**

The chatbot offered a range of benefits during its deployment. Its 24/7 availability ensured round-the-clock assistance, enhancing customer engagement and accessibility. This resulted in improved customer satisfaction as users could access information at any time without waiting for human assistance. Swift information retrieval was another advantage, enabling users to obtain property details, pricing, and availability without the delays associated with human interaction.

The efficiency of customer service was enhanced as the chatbot took care of routine inquiries, allowing human agents to dedicate more time to handling complex tasks and addressing unique customer needs. Additionally, the chatbot's ability to provide personalized recommendations based on user preferences elevated the overall user experience, making property searches more efficient and tailored.

One of the significant advantages was the chatbot's contribution to increased lead generation. By analyzing user information and preferences, the chatbot generated potential leads that could be passed on to the sales team for further engagement. This streamlined the lead generation process and improved sales efficiency.

Furthermore, the integration of the chatbot led to cost savings for the organization. Automating repetitive tasks reduced the need for additional customer support staff, resulting in optimized resource allocation and lowered operational costs. In retrospect, the chatbot's advantages had a positive impact on various aspects of customer interaction, service efficiency, and overall business performance.

## **Disadvantages of the Chatbot**

While the chatbot offered several advantages, it also came with certain drawbacks. One limitation was its limited understanding of complex or nuanced queries, which could lead to frustration for users seeking detailed information. Compared to human agents, the chatbot could provide a different level of emotional support or understanding, potentially leaving some users feeling unsatisfied.

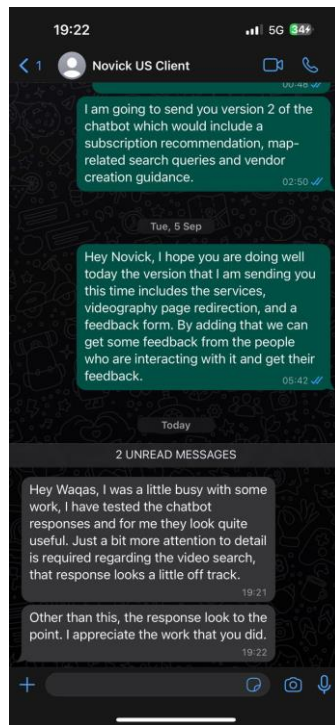
Technical issues were another concern, as glitches or malfunctions in the chatbot's functionality could negatively impact the user experience and erode trust. The chatbot's effectiveness relied heavily on the quality of its NLP models, making ongoing refinement necessary to maintain its performance standards.

Privacy concerns arose as well. Users might have been hesitant to share personal information with the chatbot, fearing potential breaches of privacy and security. This posed a challenge in establishing trust between users and the technology.

Moreover, there was a learning curve associated with the chatbot's adoption. Users needed time to adapt to the chatbot's conversational style and interaction patterns, potentially leading to initial confusion or resistance. In summary, while the chatbot provided valuable benefits, these disadvantages highlighted areas where careful consideration and improvement were required to ensure a well-rounded user experience.

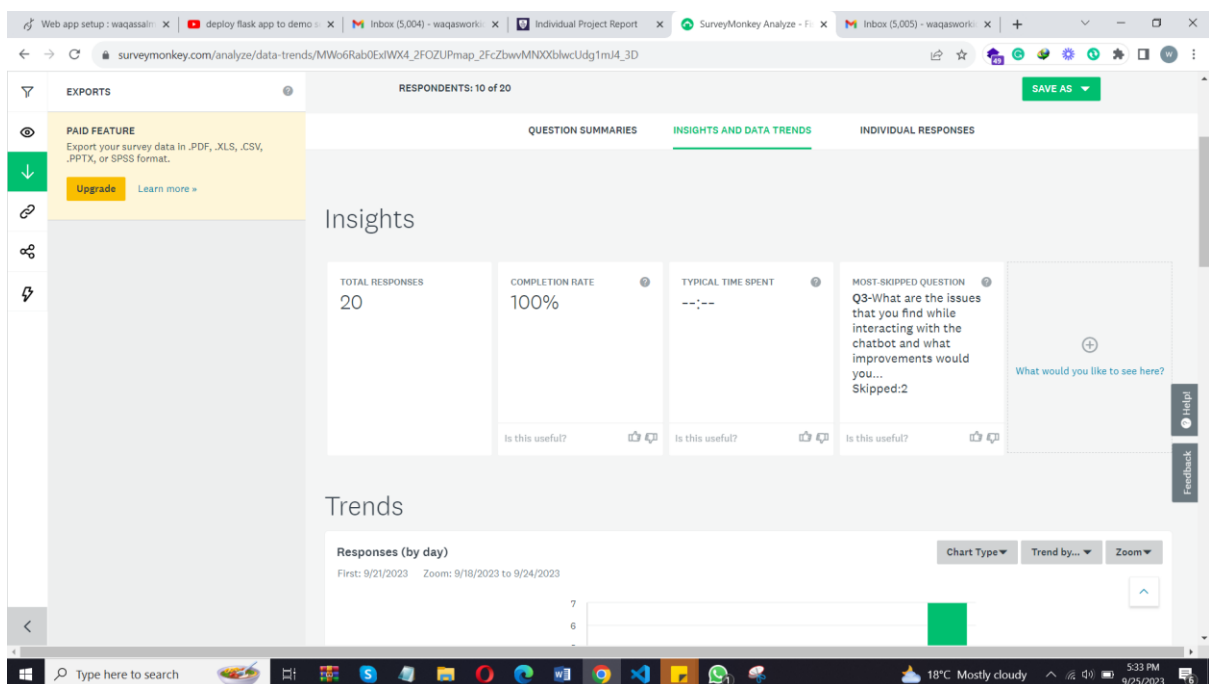
## **Discussion**

The implementation of a chatbot within the framework of Fisheye Real Estate Stock Footage Productions proved to be a valuable asset. The potential of the chatbot to enhance customer engagement and support was acknowledged by the client, the conversation is as follows:



*Fig 6.1: Conversation with the Client regarding Chatbot Functionality*

The above conversation indicates the satisfaction level of the client in terms of overall chatbot functionality. The client suggested improving the chatbot response to video queries which have been considered to be worked upon.



*Fig 6.2: Survey Monkey Insights*

Additionally, the responses on the Survey Monkey Feedback Form were recorded at the back end. In total, 20 responses were received as shown in Fig 6.2 and their distribution is as follows:

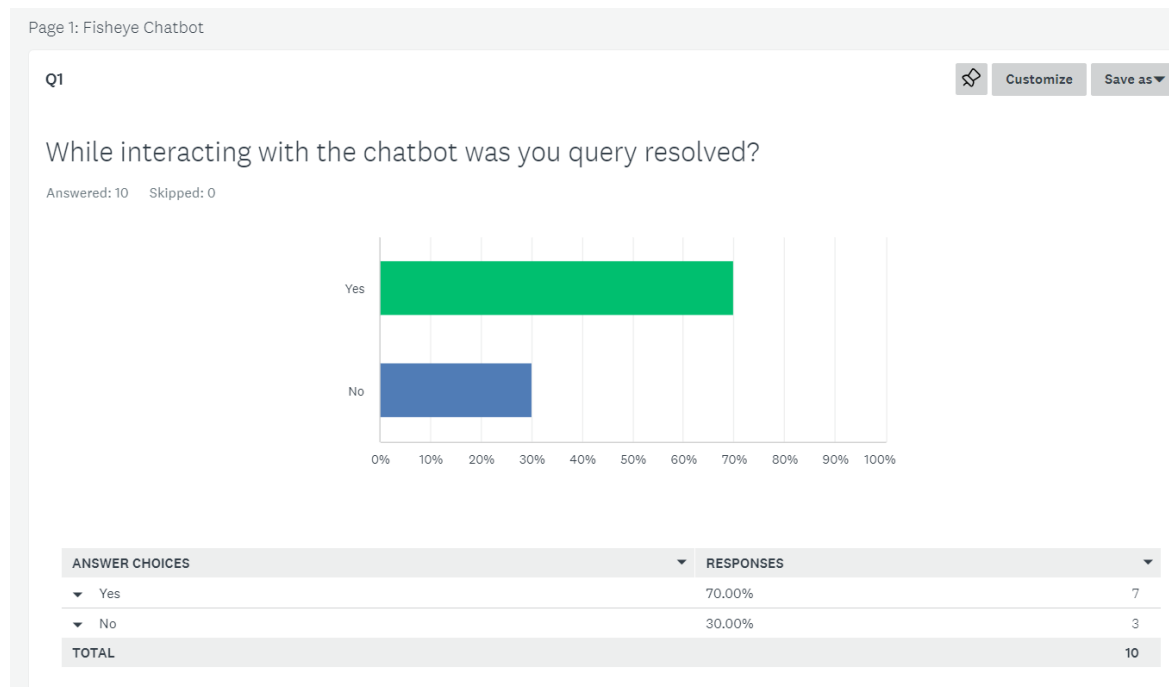


Fig 6.3: Statistical Preview of User Response to Feedback Questionnaire - Q1

In response to being asked whether their query was resolved by the chatbot or not, approximately 70% of the users agreed that while interacting the chatbot, their query was successfully resolved while 30% had the other opinion as shown in Fig 6.3.

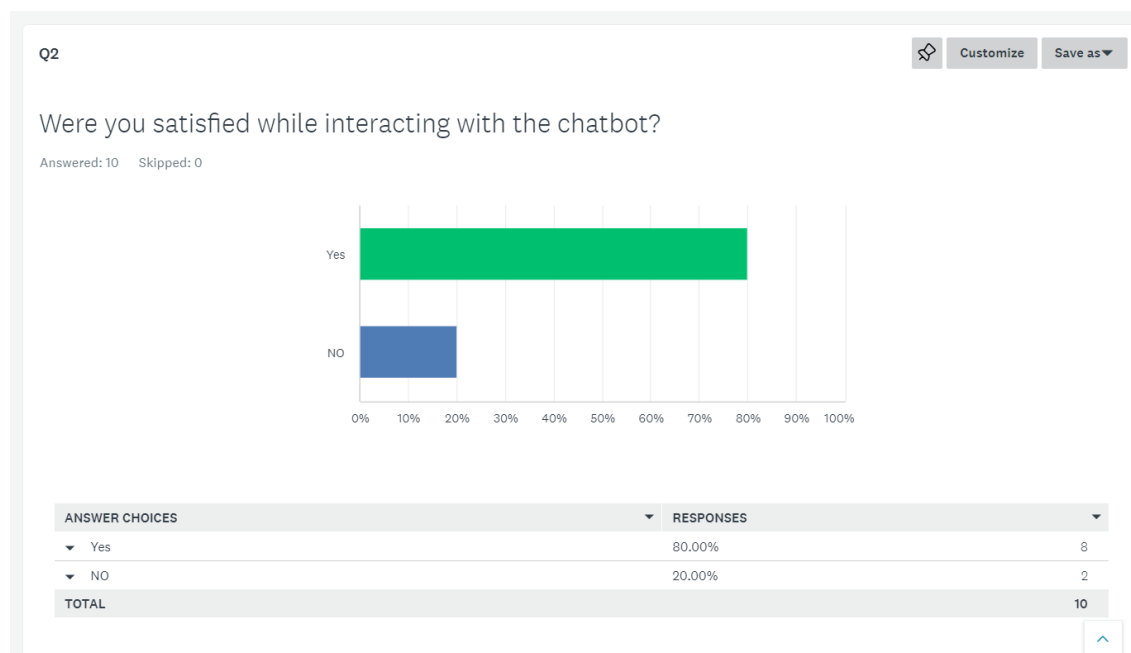
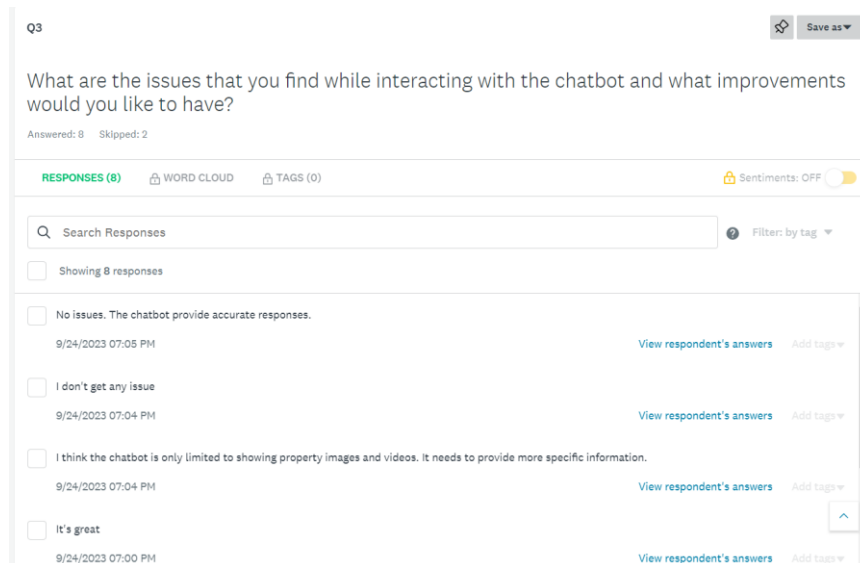


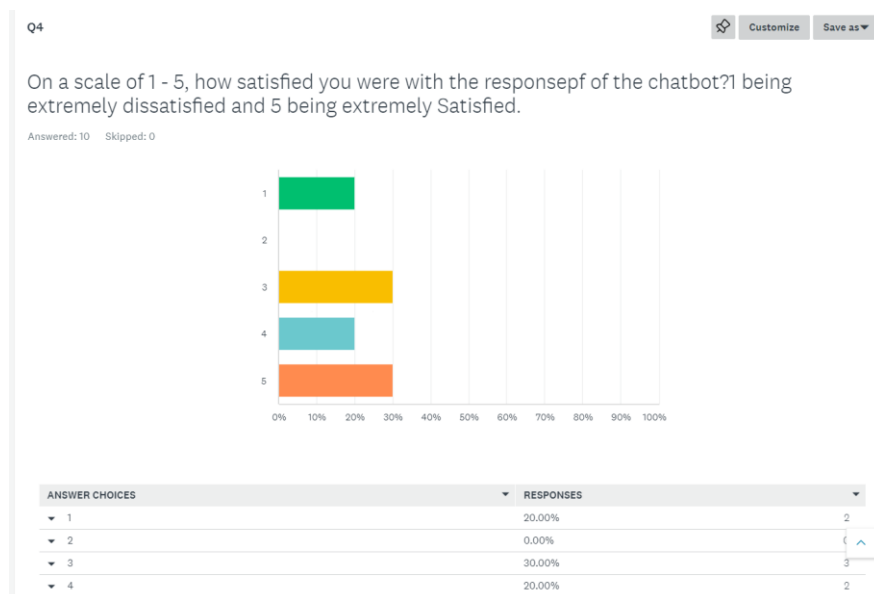
Fig 6.4: Statistical Preview of User Response to Feedback Questionnaire - Q2

In the second part of the questionnaire, chatbot users were asked that whether they were satisfied while interacting with the chatbot or not? In response, more than 80% of the users responded in “Yes” which indicates that they were indeed satisfied while interacting with the chatbot.



*Fig 6.5: User Response to Open-Ended Questions like Q3*

The above figure shows the snapshot of Q3 of the survey that was kept open-ended to provide users with an opportunity to express their concerns without limiting them to options. Therefore, in response to Q3, various open-ended answers were received suggesting that what particular issues made the users concerned while interacting with the chatbot and how those issues can be minimised.



*Fig 6.6: Statistical Preview of User Response to Feedback Questionnaire - Q4*

For the last question, users were asked to rate the chatbot on a scale of 1 to 5 and therefore, Fig 6.6 identifies the range of responses in which 50% of the people selected the option 4 and 5. This indicates that most users are satisfied while interacting with the chatbot, however, there is room to improve the chatbot functionality by addressing the issues highlighted by the users in response to Q3.

Based on the above discussion, it can be determined that certain advantages of the chatbot were vital in ensuring user satisfaction. For instance, the chatbot acted as a 24/7 virtual assistant, providing immediate access to information beyond traditional working hours. This accessibility not only satisfied client demands but also left a positive impression of the company's commitment to customer service.

Moreover, the chatbot efficiently handled routine and repetitive tasks, freeing up valuable time for more complex and strategic roles. For instance, it assisted in providing accurate visual information of the property locations, collecting basic client information, and providing automated responses to frequently asked questions. This automation not only saved time but also reduced the risk of human errors, ensuring a consistently high level of service.

The chatbot also served as a reliable source of data. It helped subscription clients to watch images and videos of the latest property locations. Additionally, it seamlessly integrated with the company's databases, retrieving information on demand and ensuring that all stakeholders had access to the most current data, enhancing decision-making processes.

Furthermore, the chatbot played a crucial role in lead generation and customer profiling. By engaging in conversations with potential clients and gathering their preferences and requirements, the chatbot contributed to creating detailed client profiles. This information was invaluable for tailoring marketing strategies, identifying potential leads, and providing personalized image & video recommendations. The chatbot also assisted in gathering contact information and initiating follow-up communications, nurturing leads into loyal customers.

In terms of cost-efficiency, the chatbot offered a compelling advantage. It required minimal maintenance, as updates and improvements were implemented through software updates. Additionally, it handled a high volume of inquiries simultaneously, reducing the need for additional staff during peak periods. This scalability ensured that the organization efficiently managed fluctuations in customer demand without incurring significant operational costs.

Lastly, the chatbot served as a data analytics tool. It gathered and analyzed customer interactions, identifying trends and patterns in client behaviour and preferences. This data-driven approach informed strategic decision-making, marketing campaigns, and

product/service offerings. By harnessing the power of data analytics, the company gained a competitive edge in the dynamic real estate market.

The integration of a chatbot into the operations of Fisheye Real Estate Stock Footage Productions proved to be a valuable asset. It enhanced client engagement, optimizing operational efficiency, and leveraged data-driven insights. Its immediate access to information, automation of routine tasks, facilitation of lead generation and profiling, cost-effectiveness, and data analytics capabilities underscored its significance as a strategic tool for the organization's growth and competitiveness.

## **Conclusion and Recommendations**

The implementation of an AI-powered chatbot for Fisheye has proven to be a promising venture. The chatbot's ability to engage real estate clients, streamline interactions, and provide valuable insights about property locations, can undoubtedly bring a competitive edge to the real estate industry. As AI technologies continue to advance, further research and development in this area hold the potential to revolutionize how clients interact with real estate businesses, providing them with a more efficient and personalized experience. Fisheye, as well as other real estate setups in the industry, should consider integrating AI-powered chatbots into their business strategies to adapt to the evolving demands of the market and stay ahead in the digital age.

Moreover, considering the successful implementation and value addition of the chatbot at Fisheye Real Estate Stock Footage Productions, several future recommendations can further enhance the chatbot's capabilities:

**Advanced Natural Language Processing (NLP):** To improve the chatbot's conversational abilities and understanding of nuanced queries, Fisheye could invest in advanced NLP models. These models can handle complex language structures and provide more accurate responses, enhancing user satisfaction.

**Multilingual Support:** As Fisheye Real Estate expands into global markets, implementing multilingual support within the chatbot can cater to a broader audience. This would involve training the chatbot in different languages and ensuring it can seamlessly switch between them based on user preferences.

**Voice Integration:** Integrating voice recognition capabilities into the chatbot can offer an additional communication channel for users. Voice-enabled chatbots can assist users who prefer speaking over typing, making the interaction more accessible and convenient.

**Personalization:** Enhancing the chatbot's ability to deliver personalized experiences is essential. By leveraging user data and preferences, the chatbot can recommend personalized property images and footage, send personalized notifications, and tailor responses to individual user needs.

**Integration with Virtual Tours:** As the real estate industry increasingly relies on virtual property tours, integrating the chatbot with virtual tour platforms can be beneficial. The chatbot can guide users through virtual tours, answer questions about properties, and provide relevant information during the tours.

**Data Security and Privacy:** Ensuring robust data security and privacy measures is crucial. Fisheye Real Estate should continuously update the chatbot's security protocols to protect sensitive customer data and comply with data privacy regulations such as GDPR.

**Feedback Mechanisms:** Currently the chatbot is integrated with an external Survey Monkey feedback form. Fisheye can improve the feedback mechanisms by integrating such forms using an in-built interface where the users do not redirect towards external links.

**Seamless CRM Integration:** Integrating the chatbot with the company's CRM system can streamline lead management. The chatbot should be able to capture and update customer information, ensuring a seamless handoff to human agents when necessary.

**Continuous Training and Updates:** Regularly updating the chatbot's knowledge base and training data is essential to keep it relevant and accurate. Fisheye should invest in ongoing training to ensure the chatbot stays up-to-date with industry trends and changes.

**Marketing and Promotion:** Promoting the chatbot to clients and potential customers is crucial. Fisheye can incorporate the chatbot into its marketing strategy, highlighting its availability and capabilities on the website and through marketing channels.

**Performance Analytics:** Implementing robust analytics tools to monitor the chatbot's performance is essential. Tracking metrics such as user engagement, response times, and conversion rates can provide insights for further improvements.

**Scaling Capabilities:** As the organization grows, the chatbot should be designed to scale accordingly. Fisheye should ensure that the chatbot can handle increased user loads and adapt to changing business needs.

Incorporating these recommendations can help Fisheye Real Estate Stock Footage Productions continue to leverage its chatbot effectively, providing enhanced customer

experiences, improving operational efficiency, and staying competitive in the real estate market.

## References

- Abioye, S.O., Oyedele, L.O., Akanbi, L., Ajayi, A., Delgado, J.M.D., Bilal, M., Akinade, O.O. and Ahmed, A., 2021. Artificial intelligence in the construction industry: A review of present status, opportunities and future challenges. *Journal of Building Engineering*, 44, p.103299.
- Amato, A., Cozzolino, G. and Ferraro, A., 2021. Artificial intelligent chatbot for food related question. *Advances on P2P, Parallel, Grid, Cloud and Internet Computing: Proceedings of the 15th International Conference on P2P, Parallel, Grid, Cloud and Internet Computing (3PGCIC-2020)*, 15, pp. 235-240.
- Bauer, K., von Zahn, M. and Hinz, O., 2023. Expl (AI) ned: The impact of explainable artificial intelligence on users' information processing. *Information Systems Research*.
- Bilgin, T.T. and Yavuz, E., 2022, December. Integrating A Dialogue Tree Based Turkish Chatbot into an Open Source Python Coding Editor. In 2022 3rd International Informatics and Software Engineering Conference (IISEC) (pp. 1-5). IEEE.
- Brownlee, J., 2019. Develop deep learning models on Theano and TensorFlow using keras. *Journal of Chemical Information and Modeling*, 53(9), pp.1689-1699.
- Cao, T.D. and Nguyen, Q.H., 2021. An approach for building effective real Estate Chatbots in Vietnamese. *Soft Computing for Biomedical Applications and Related Topics*, pp.221-229.
- Castle-Green, T., Reeves, S., Fischer, J.E. and Koleva, B., 2020, Decision trees as sociotechnical objects in chatbot design. *2nd Conference on Conversational User Interfaces* (pp. 1-3).
- Chin, H., Lima, G., Shin, M., Zhunis, A., Cha, C., Choi, J. and Cha, M., 2023. User-chatbot conversations during the COVID-19 pandemic: a study based on topic modelling and sentiment analysis. *Journal of Medical Internet Research*, 25, p.e. 40922.
- Chopra, D. and Khurana, R., 2023. Introduction to Machine Learning with Python. *Bentham Science Publishers*.
- Dash, M. and Bakshi, S., 2019. An exploratory study of customer perceptions of usage of chatbots in the hospitality industry. *International Journal on Customer Relations*, 7(2), pp.27-33.

Fisheye., 2023. Available at: <https://fisheye.video/>

Galitsky, B. and Galitsky, B., 2021. A virtual social promotion chatbot with persuasion and rhetorical coordination. *Artificial Intelligence for Customer Relationship Management: Solving Customer Problems*, pp.129-179.

James, B.V., Joseph, D. and Daniel, N., 2023. Young adults' experience of housing and real estate chatbots in India: effort expectancy moderated model. *International Journal of Housing Markets and Analysis*.

Javed, A.R., Ahmed, W., Pandya, S., Maddikunta, P.K.R., Alazab, M. and Gadekallu, T.R., 2023. A survey of explainable artificial intelligence for smart cities. *Electronics*, 12(4), p.1020.

Jenneboer, L., Herrando, C. and Constantinides, E., 2022. The impact of chatbots on customer loyalty: A systematic literature review. *Journal of theoretical and applied electronic commerce research*, 17(1), pp.212-229.

Kathirvelu, M., Janaranjani, A., Pranav, A.N. and Pradeep, R., 2022. Voice Recognition Chat bot for Consumer Product Applications. *IEEE International Conference on Data Science and Information System (ICDSIS)*, pp. 1-5.

Kaur, T. and Solomon, P., 2022. A study on automated property management in commercial real estate: a case of India. *Property Management*, 40(2), pp.247-264.

Knott, A., 2023. Job sharing between human professionals and chatbots: How should 'handovers' happen? *Journal of AI, Robotics & Workplace Automation*, 2(2), pp.121-130.

Kumar, L.A. and Renuka, D.K., 2023. *Deep Learning Approach for Natural Language Processing, Speech, and Computer Vision: Techniques and Use Cases*. CRC Press.

Kushwaha, N.S. and Singh, P., 2022. Artificial Intelligence based Chatbot: A Case Study. *Journal of Management and Service Science (JMSS)*, 2(1), pp.1-13.

Le, A.C. and Huynh, V.N., 2022, October. Towards a Human-like Chatbot using Deep Adversarial Learning. In *2022 14th International Conference on Knowledge and Systems Engineering (KSE)* (pp. 1-5). IEEE.

Le, A.C., 2023. Improving Chatbot Responses with Context and Deep Seq2Seq Reinforcement Learning. *Research Square*.

- Lee, S.B., 2020. Chatbots and communication: the growing role of artificial intelligence in addressing and shaping customer needs. *Business Communication Research and Practice*, 3(2), pp.103-111.
- Lin, C.C., Huang, A.Y. and Yang, S.J., 2023. A review of AI-driven conversational chatbot implementation methodologies and challenges (1999–2022). *Sustainability*, 15(5), p.4012.
- Listl, F.G., Fischer, J. and Weyrich, M., 2021, September. Towards a Simulation-based conversational assistant for the operation and engineering of production plants. *26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, (pp. 1-4.
- Longo, C.F., Riela, P.M., Santamaria, D.F., Santoro, C. and Lieto, A., 2023. A framework for cognitive chatbots based on abductive–deductive inference. *Cognitive Systems Research*, 81, pp.64-79.
- Longoni, C. and Cian, L., 2022. Artificial intelligence in utilitarian vs. hedonic contexts: The “word-of-machine” effect. *Journal of Marketing*, 86(1), pp.91-108.
- Mah, P.M., Skalna, I., Muzam, J. and Song, L., 2022. Analysis of Natural Language Processing in the FinTech Models of Mid-21st Century. *Journal of Information Technology and Digital World*, 4, pp.183-211.
- McCord, M., 2022. A Review of the methods, applications, and challenges of adopting artificial intelligence in the property assessment office. *International Association of Assessing Officers (IAAO)*, pp. 15-16.
- Meshram, S., Naik, N., Megha, V.R., More, T. and Kharche, S., 2021. Conversational AI: Chatbots. *International Conference on Intelligent Technologies (CONIT)*. pp. 1-6.
- Miljkovic, I., Shlyakhetko, O. and Fedushko, S., 2023. Real Estate App Development Based on AI/VR Technologies. *Electronics*, 12(3), p.707.
- Nawaz, N. and Saldeen, M.A., 2020. Artificial intelligence chatbots for library reference services. *Journal of Management Information & Decision Sciences*, 23.
- Pandey, S. and Sharma, S., 2023. A comparative study of retrieval-based and generative-based chatbots using Deep Learning and Machine Learning. *Healthcare Analytics*, p.100198.

- Petralba, J., 2020. Wordnet Semantic Relations in a Chatbot. *Recoletos Multidisciplinary Research Journal*, 8(2), pp.15-34.
- Sağlam, R.B. and Nurse, J.R., 2020. Is your chatbot GDPR compliant? Open issues in agent design. *2nd Conference on Conversational User Interfaces*. pp. 1-3.
- Seagraves, P., 2023. Real Estate Insights: Is the AI revolution a real estate boon or bane?. *Journal of Property Investment & Finance*.
- Shashavali, D., Vishwjeet, V., Kumar, R., Mathur, G., Nihal, N., Mukherjee, S. and Patil, S.V., 2019. Sentence similarity techniques for short vs variable length text using word embeddings. *Computación y Sistemas*, 23(3), pp.999-1004.
- Singh, J., 2023. *Natural Language Processing in the Real World: Text Processing, Analytics, and Classification*. CRC Press.
- Solanki, M.S., 2021. An analysis of chatbot design techniques. *ACADEMICIA: An International Multidisciplinary Research Journal*, 11(10), pp.829-836.
- Souali, K., Rahmaoui, O., Ouzzif, M. and El Haddioui, I., 2019. Recommending moodle resources using chatbots. *15th International Conference on Signal-Image Technology & Internet-Based Systems (SITIS)*, pp. 677-680.
- Spring, M., Faulconbridge, J. and Sarwar, A., 2022. How information technology automates and augments processes: Insights from Artificial- Intelligence- based systems in professional service operations. *Journal of Operations Management*, 68(6-7), pp.592-618.
- Stoilova, E., 2021. AI chatbots as a customer service and support tool. *ROBONOMICS: The Journal of the Automated Economy*, 2, pp.21-21.
- Survey Monkey, 2023. Available at: <https://www.surveymonkey.com/r/PGLDGYW>
- Thomas, A., 2023. Facilitating Human-Robot Interaction through Emotional Control: A Case Study of Optimus and Large Language Model Chatbots.
- Torous, J., Bucci, S., Bell, I.H., Kessing, L.V., Faurholt- Jepsen, M., Whelan, P., Carvalho, A.F., Keshavan, M., Linardon, J. and Firth, J., 2021. The growing field of digital psychiatry: current evidence and the future of apps, social media, chatbots, and virtual reality. *World Psychiatry*, 20(3), pp.318-335.

Treleaven, P., Barnett, J., Knight, A. and Serrano, W., 2021. Real estate data marketplace. *AI and Ethics*, 1, pp.445-462.

Völkel, S.T., Schoedel, R., Kaya, L. and Mayer, S., 2022, April. User perceptions of extraversion in chatbots after repeated use. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems* (pp. 1-18).

Yadav, R., 2022. Exploring B2B Chatbots adoption experiences: Lessons for successful implementation in Businesses.

Zhang, W., 2020. The Comparison Between the Tools for Named Entity Recognition. *Doctoral dissertation, Auckland University of Technology*.

Zhao, Y., Chetty, G. and Tran, D., 2022, December. Deep Learning for Real Estate Trading. In *2022 IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE)* (pp. 1-7). IEEE.

Zhu, X. and Li, Y., 2022, December. Big Data Technology in Real Estate Industry: Scenarios and Benefits. In *Big Data–BigData 2022: 11th International Conference, Held as Part of the Services Conference Federation, SCF 2022, Honolulu, HI, USA, December 10–14, 2022, Proceedings* (pp. 70-79). Cham: Springer International Publishing.

Zhu, H., 2023. *Understanding Customers in AI-empowered Financial Advisory Systems and Services: An Interdisciplinary Study of Robo-advisors* (Doctoral dissertation, KTH Royal Institute of Technology).

# Appendix

## Documenting the Code Development for Chatbot

Following is the coding used during the chatbot development process:

### 1. App.py

```
import nltk
import spacy
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
import pickle
from nltk.corpus import wordnet
import numpy as np
import requests

from keras.models import load_model
model = load_model('model.h5')
import json
import random
intents = json.loads(open('data.json').read())
type_response_API = requests.get('https://content.fisheye.video/api/video_types')
type_data_file = type_response_API.text
type = json.loads(type_data_file)
location_response_API = requests.get('https://content.fisheye.video/api/locations')
locations_data_file = location_response_API.text
locations = json.loads(locations_data_file)
map_response_API = requests.get('https://content.fisheye.video/api/map')
map_data_file = map_response_API.text
maps = json.loads(map_data_file)
words = pickle.load(open('texts.pkl','rb'))
classes = pickle.load(open('labels.pkl','rb'))
# A simple grammar to create tree
grammar = ""
    Subject: {<NNP><NNP>}d
    Action: {<MD><VB>}
    Object: {<DT><NN>}
    When: {<NNP><CD>}
""

action_tags = ["VB","VBD","VBG","VBN","JJ"]
map_tags = ["map","maps","graphical","location"]

import nltk.corpus

def map(sent_tree):
```

```

    for tag_words in sent_tree:
        if tag_words[0] in map_tags:
            return tag_words[0]

def action_taken(sent_tree):
    for tag_words in sent_tree:
        if tag_words[1] in action_tags:
            return tag_words[0]

nlp = spacy.load("en_core_web_sm")

def extract_locations(text):
    document = nlp(text)

    location = []

    for entities in document.ents:
        if entities.label_ == "GPE":
            location.append(entities.text)

    return location

def cleaning_up_sentence(sentence):
    sent_words = nltk.word_tokenize(sentence)
    return sent_words

# return bag of words array: 0 or 1 for each word in the bag that exists in the sentence

def bag_of_words(sentence, words, show_details=True):
    sentence_words = cleaning_up_sentence(sentence)
    # tag the words
    words_tagged = nltk.pos_tag(sentence_words)
    # Creating a tree
    parser = nltk.RegexpParser(grammar)
    tree = parser.parse(words_tagged)
    data = extract_locations(sentence)
    print("Action:", action_taken(tree))
    print("Location:", data)
    # Bag of wordings
    bag = [0]*len(words)
    # Searching words in the trained data for a match
    for sentence in sentence_words:
        for i, words_list in enumerate(words):
            # Check here for new words like near close
            if words_list == sentence:
                bag[i] = 1
                if show_details:

```

```

        print ("found in bag: %s" % words_list)

    return(np.array(bag))
# Searching for Map Keywords
def map_words(sentence):
    map_value = False
    # tokenize the pattern
    sentence_words = cleaning_up_sentence(sentence)
    # POS tag the words
    words_tagged = nltk.pos_tag(sentence_words)
    # Create tree
    parser = nltk.RegexpParser(grammar)
    tree = parser.parse(words_tagged)
    # print(tree)
    # data = map(tree)
    # print(data)
    for ne in tree:
        if hasattr(ne, "label"):
            value = [c[0] for c in ne]
            for maps in map_tags:
                if maps in value:
                    map_value = True
    return map_value

def predicting_class(sentence, model):
    words_bag = bag_of_words(sentence, words, show_details=False)
    map_value = map_words(sentence)
    result = model.predict(np.array([words_bag]))[0]
    ERROR_THRESHOLD = 0.25
    results = [[i,r] for i,r in enumerate(result) if r>ERROR_THRESHOLD]
    # sort by probability
    results.sort(key=lambda x: x[1], reverse=True)
    return_listing = []
    for r in results:
        return_listing.append({"intent": classes[r[0]], "probability": str(r[1]), "maps": map_value})
    print(return_listing)
    return return_listing

def getResponse(ints, intents_json, locations_json, type_json, map_json):
    tag = ints[0]['intent']
    map_value = ints[0]['maps']
    print(map_value)
    list_of_intents = intents_json['intents']
    list_of_locations = locations_json['data']
    list_of_type = type_json['data']
    list_of_map = map_json['data']
    if map_value == True:

```

```

print("We are here for map search")
# CURRENTLY WORKING HERE
for l in list_of_map:
    location = cleaning_up_sentence(l['location'])
    print(location, " ////////// ", tag)
    if(tag in location):
        print(l)
        result = str(l['video_id'])
        result = '<p>Here are some results related to your search shown on a map for
precise location:</p><br>'+<a target="_blank"
href="https://fisheye.video/maps/"+result+">https://fisheye.video/maps/"+result+'</a>'
        break
    else:
        result = '<p>Sorry, We can not find any map preview according to your
search</p>'
    return result
else:
    for j in list_of_locations:
        if(j['name'] == tag):
            result = str(j['id'])
            result = '<p>You can find some results related to your search on the following
page:</p><br>'+<a target="_blank" href="https://fisheye.video/Video-
category?location="+result+">https://fisheye.video/Video-category?location="+result+'</a>'
            break
    for k in list_of_type:
        if(k['name'] == tag):
            result = str(k['id'])
            result = '<p>You can find some results related to your search on the following
page:</p><br>'+<a target="_blank" href="https://fisheye.video/Video-
category?type="+result+">https://fisheye.video/Video-category?type="+result+'</a>'
            break
    for i in list_of_intents:
        if(i['tag'] == tag):
            result = random.choice(i['responses'])
            break
    return result

def chatbot_result(msg):
    message = lemmatizer.lemmatize(msg.lower())
    ints = predicting_class(message, model)
    result = getResponse(ints, intents, locations, type, maps)
    return result

from flask import Flask, render_template, request

app = Flask(__name__)
app.static_folder = 'static'

```

```
@app.route("/")
def home():
    return render_template("Chatbot.html")
```

```
@app.route("/get")
def get_bot_response():
    userText = request.args.get('msg')
    return chatbot_result(userText)
```

```
if __name__ == "__main__":
    app.run()
```

## 2. Training.py

```
import nltk
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
import json
import pickle
import requests
```

```
import numpy as np
from keras.models import Sequential
from keras.layers import Dense, Activation, Dropout
from keras.optimizers import SGD
import random
```

```
words=[]
more_new_words = []
new_words = []
map_words = []
location_words = ["near","in","close","within","inside","at","on","part of"]
class_required = []
docs = []
ignoring_words = ['?', '!', ',', '.']
data_file = open('data.json').read()
intents = json.loads(data_file)
response_API = requests.get('https://content.fisheye.video/api/locations')
map_API = requests.get('https://content.fisheye.video/api/map')
locations_data_file = response_API.json()
map_data_file = map_API.json()
type_response_API = requests.get('https://content.fisheye.video/api/video_types')
type_data_file = type_response_API.json()
```

```
for location in locations_data_file['data']:
```

```
    #tokenize each word
    x = nltk.word_tokenize(location['name'])
```

```

new_words.extend(x)
print('Locations:',x)
docs.append((x, location['name']))

# add to our class_required list
if location['name'] not in class_required:
    class_required.append(location['name'])

for type in type_data_file['data']:

    #tokenize each word
    y = nltk.word_tokenize(type['name'])
    more_new_words.extend(y)
    print('Type:',y)
    docs.append((y, type['name']))

    # add to our class_required list
    if type['name'] not in class_required:
        class_required.append(type['name'])

for map in map_data_file['data']:

    #tokenize each word
    z = nltk.sent_tokenize(map['location'])
    map_words.append(map['location'])
    print('Map Location:',z)
    #add docs in the corpus
    docs.append((z, map['location']))

    # add to our class_required list
    if map['location'] not in class_required:
        class_required.append(map['location'])

for intent in intents['intents']:
    for pattern in intent['patterns']:

        bag_words = nltk.sent_tokenize(pattern)
        words.extend(bag_words)
        docs.append((bag_words, intent['tag']))

        if intent['tag'] not in class_required:
            class_required.append(intent['tag'])

words = [lemmatizer.lemmatize(w.lower()) for w in words if w not in ignoring_words]
New_words = [lemmatizer.lemmatize(x.lower()) for x in new_words if x not in
ignoring_words]
More_New_words = [lemmatizer.lemmatize(y.lower()) for y in more_new_words if y not in
ignoring_words]

```

```

More_New_words_2 = [lemmatizer.lemmatize(z.lower()) for z in map_words if z not in
ignoring_words]
words =
sorted(list(set(words))+list(set(New_words))+list(set(More_New_words))+list(set(More_New
_words_2)))
print("Old Words", words)
# sort class_required
class_required = sorted(list(set(class_required)))
# docs = combination between patterns and intents
print (len(docs), "docs")
# class_required = intents
print (len(class_required), "class_required", class_required)
# words = all words, vocabulary
print (len(words), "unique lemmatized words", words)

pickle.dump(words,open('texts.pkl','wb'))
pickle.dump(class_required,open('labels.pkl','wb'))

# create our training data
training = []
# create an empty array for our output
output_empty = [0] * len(class_required)
# training set, bag of words for each sentence
for doc in docs:
    # initialize our bag of words
    bag = []
    # list of tokenized words for the pattern
    pattern_words = doc[0]
    # lemmatize each word - create base word, in attempt to represent related words
    pattern_words = [lemmatizer.lemmatize(word.lower()) for word in pattern_words]
    # create our bag of words array with 1, if word match found in current pattern
    for w in words:
        bag.append(1) if w in pattern_words else bag.append(0)

    # output is a '0' for each tag and '1' for current tag (for each pattern)
    output_row = list(output_empty)
    output_row[class_required.index(doc[1])] = 1

    training.append([bag, output_row])
# shuffle our features and turn into np.array
random.shuffle(training)
training = np.array(training)
# create train and test lists. X - patterns, Y - intents
train_x = list(training[:,0])
train_y = list(training[:,1])
print("Training data created")

```

```

# Create model - 3 layers. First layer 128 neurons, second layer 64 neurons and 3rd output
layer contains number of neurons
# equal to number of intents to predict output intent with softmax
model = Sequential()
model.add(Dense(128, input_shape=(len(train_x[0]),), activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(len(train_y[0]), activation='softmax'))

# Compile model. Stochastic gradient descent with Nesterov accelerated gradient gives
good results for this model
# sgd = SGD(lr=0.01, decay=1e-6, momentum=0.9, nesterov=True)
model.compile(loss='categorical_crossentropy', metrics=['accuracy'])

#fitting and saving the model
hist = model.fit(np.array(train_x), np.array(train_y), epochs=200, batch_size=5, verbose=1)
model.save('model.h5', hist)

print("model created")

```

### 3. Data.json

```

{
  "intents": [
    {
      "tag": "greeting",
      "patterns": [
        "Hi there",
        "Is anyone there?",
        "Hey",
        "Hello",
        "Good day",
        "Hi"
      ],
      "responses": [
        "Hi there, how can I help?"
      ],
      "context": [
        ""
      ]
    },
    {
      "tag": "info",
      "patterns": [
        "How are you",
        "How are you doing",
        "How's everything",

```

```

        "How's it going",
        "What's up"
    ],
    "responses": [
        "Good, how can I help?"
    ],
    "context": [
        ""
    ]
},
{
    "tag": "creator",
    "patterns": [
        "Creator",
        "I want to become a creator",
        "Videography",
        "register as a creator"
    ],
    "responses": [
        "You can register yourself here as a creator to earn money: <a
href='https://fisheye.video/vendorRegister'>Creators</a>"
    ],
    "context": [
        ""
    ]
},
{
    "tag": "about",
    "patterns": [
        "Tell me about fisheye website",
        "how you work",
        "What does fisheye do",
        "How fisheye works",
        "How can fisheye help me",
        "How you could help me?",
        "What you can do?",
        "What help can you provide?",
        "How can you be helpful?",
        "What support is offered"
    ],
    "responses": [
        "Fisheye specialize in providing Creative Development to the clients. With
fisheye's team of experts, rest assured that your visual content will be nothing short of
exceptional."
    ],
    "context": [
        ""
    ]
}

```

```

},
{
  "tag": "Subscripitons",
  "patterns": [
    "Tell me about your packages",
    "Information about Subscriptions",
    "subscriptions",
    "packages",
    "fisheye packages",
    "fisheye subscriptions"
  ],
  "responses": [
    "We offer 3 subscription plans at the moment Commercial <h3>REQUEST MAP  

    LOCATIONS</h3> <p>Per month $999 Billed annually</p> <br> <ul class='msg-ul'><li  

    >One Simple License </li><li >Unlimited Downloads </li><li >Unlimited use </li><li >Yours  

    Forever </li><!--><li >Discounts on our services </li><li >Request Map Locations </li><li  

    >Ungraded Clips </li><li >4K+ Clips </li><li >HD Clips </li><li >Color Graded Clips </li><!--></ul> <br>
    Here is the link for more details about the packages: <a  

    href='https://fisheye.video/subscribe'>Subscription Plans</a>"
  ],
  "context": [
    ""
  ]
},
{
  "tag": "services",
  "patterns": [
    "services",
    "fisheye services",
    "how fisheye can help me",
    "fisheye usage",
    "your services"
  ],
  "responses": [
    "Fisheye specialize in providing Creative Development to the clients. With  

    fisheye's team of experts, rest assured that your visual content will be nothing short of  

    exceptional. you can find more information at: https://fisheye.video/services"
  ],
  "context": [
    ""
  ]
},
{
  "tag": "goodbye",
  "patterns": [
    "Bye",
    "See you later",
    "Goodbye",

```

```

        "Nice chatting to you, bye",
        "Till next time"
    ],
    "responses": [
        "Feedback Time <br><br> Thank you for interacting with us! But before you leave,
        <br> we would like to have your feedback which is invaluable for us in improving our
        customer service <br><br> <a href='https://www.surveymonkey.com/r/PGLDGYW'
        target='_blank'>https://www.surveymonkey.com/r/PGLDGYW</a>"
    ],
    "context": [
        ""
    ]
},
{
    "tag": "thanks",
    "patterns": [
        "Thanks",
        "Thank you",
        "That's helpful",
        "Awesome, thanks",
        "Thanks for helping me"
    ],
    "responses": [
        "Happy to help!",
        "Any time!",
        "My pleasure"
    ],
    "context": [
        ""
    ]
},
{
    "tag": "noanswer",
    "patterns": ["", ""],
    "responses": [
        "Sorry, can't understand you",
        "Please give me more info",
        "Not sure I understand"
    ],
    "context": [
        ""
    ]
}
]
}

```